Tugas Proyek dalam Kelompok
Audio Deepfake Anomaly Detection Harness
(ADADeH)
II4050 Rekayasa Sistem Multimedia

Oleh

Samuel Chris M.B.S. 18223011
Carlen Asadel Axelle 18223017
Ananda Fajrul Zidan 18223067



PROGRAM STUDI SISTEM DAN TEKNOLOGI INFORMASI
SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA
INSTITUT TEKNOLOGI BANDUNG
November 2026

DAFTAR ISI

DAFTAR ISI iii
DAFTAR GAMBAR vii
DAFTAR TABEL ix
DAFTAR PERSAMAAN xi
DAFTAR ALGORITMA xiii
DAFTAR KODE PROGRAM xv
DAFTAR SINGKATAN xvii
DAFTAR SIMBOL xix

BAB I PENDAHULUAN **1**
I.1 Latar Belakang 1
I.2 Rumusan Masalah 2
I.3 Tujuan 2
I.4 Lingkup dan Batasan 3
I.5 Metodologi Pengembangan 3
I.6 Sistematika Penulisan 4

BAB II DASAR TEORI **5**
II.1 Data Multimedia dan Representasinya 5
II.1.1 Data Audio 5
II.1.2 Data Citra Spektrogram 6
II.1.2.1 *Mel-Spectrogram* 6
II.1.2.2 Pemrosesan Spektrogram 7
II.2 Sistem Multimedia 8
II.3 Perangkat Keras dan Perangkat Lunak 10
II.3.1 Perangkat Keras 10
II.3.1.1 Perangkat Pelatihan Model 10
II.3.1.2 Perangkat Pengembangan dan Pengujian 11
II.3.1.3 Perangkat Pengguna 11
II.3.2 Perangkat Lunak 12
II.3.2.1 Bahasa Pemrograman dan Lingkungan Pengembangan 12
II.3.2.2 Pustaka Pemrosesan Audio 13
II.3.2.3 Pustaka Pembelajaran Mendalam 13
II.3.2.4 Pustaka Evaluasi dan Visualisasi 13
II.3.2.5 Perangkat Lunak Antarmuka 14
II.3.3 Arsitektur Klien–Server dan Transportasi Data 14
II.3.4 HTTP dan HTTPS 15
II.3.5 WebSocket dan Pengelolaan Sesi 15
II.3.6 Alur Pengiriman Audio dan Penyajian Hasil 16
II.4 Teknologi-Teknologi Lainnya 17
II.4.1 Deteksi *Deepfake Audio* 17
II.4.2 CNN untuk Klasifikasi Spektrogram 17
II.4.3 Pemisahan Data untuk Pelatihan dan Evaluasi 18

II.4.4 Metrik Evaluasi dan Interpretasi Hasil 18

BAB III ANALISIS KEBUTUHAN 21
III.1 Metode Pengumpulan Data 21
III.1.1 Tahapan Studi Literatur 21
III.1.2 Hasil Studi Literatur 22
III.2 Analisis Kebutuhan Pengguna 23
III.2.1 Identifikasi Pengguna 23
III.2.2 Rumusan Kebutuhan Pengguna 23
III.2.3 Keterlacakan Kebutuhan 26
III.3 Pemilihan Alternatif Solusi 26
III.3.1 Alternatif Solusi 26
III.3.2 Kriteria Penilaian 27
III.3.3 Matriks Keputusan 27
III.3.4 Solusi Terpilih 28
III.4 Analisis Kebutuhan Sistem 28
III.4.1 Kebutuhan Fungsional 28
III.4.2 Kebutuhan Nonfungsional 29
III.4.3 Keterlacakan Kebutuhan Pengguna terhadap Kebutuhan Sistem 29

BAB IV Desain dan Implementasi 37
IV.1 Desain Sistem 37
IV.1.1 Prinsip Desain 37
IV.1.2 Alur Proses Sistem 38
IV.1.3 Rancangan Keluaran bagi Pengguna 39
IV.2 Arsitektur Sistem 39
IV.2.1 Arsitektur Klien–Server 39
IV.2.2 Arsitektur Paket Perangkat Lunak `adadeh` 42
IV.2.3 Arsitektur Model Klasifikasi CNN 42
IV.2.4 Arsitektur Konfigurasi dan Artefak Model 45
IV.3 Implementasi 46
IV.3.1 Lingkungan dan Pustaka Implementasi 46
IV.3.1.0.1 Catatan penyesuaian implementasi. . . . 46
IV.3.2 Implementasi Akuisisi Data 47
IV.3.3 Implementasi Pembagian Data 48
IV.3.4 Implementasi Pra-Pemrosesan dan Pemotongan Audio . . 49
IV.3.5 Implementasi Model CNN 51
IV.3.6 Implementasi Pelatihan Model 52
IV.3.7 Implementasi Evaluasi Model 54
IV.3.8 Implementasi Penyimpanan Artefak Model 56
IV.3.9 Implementasi Validasi dan Keamanan Audio 57
IV.3.10 Implementasi Runtime Model dan Verifikasi Integritas . . 57
IV.3.11 Implementasi Pemeriksaan Mutu Rekaman 58
IV.3.12 Implementasi Layanan, Antarmuka, dan Pelaporan 59
IV.3.13 Implementasi Evaluasi Purwarupa dan Reproduksi *Baseline* 59
IV.3.14 Pengujian Perangkat Lunak 60

IV.4 Rangkuman 61

BAB V RENCANA SELANJUTNYA 63

DAFTAR PUSTAKA 65

Lampiran A *SOURCE CODE* 67
A.1 Perangkat Lunak untuk Akuisisi Data dari Sensor Ultrasonik . . . 67
A.2 Perangkat Lunak untuk Pengolahan Data Akuisisi dan Visualisasi Hasil Pengukuran Jarak 68

DAFTAR GAMBAR

II.1 Proses Pembentukkan Spektrogram (I. McLoughlin dkk. 2026) . . . 6

III.1 Alur Penurunan Kebutuhan ADADeH 21
III.2 Tahapan Studi Literatur 22

IV.1 Alur Proses Sistem ADADeH dari Unggahan hingga Penyajian Hasil 38
IV.2 Arsitektur Klien–Server ADADeH 40
IV.3 Arsitektur Model CNN untuk Klasifikasi Mel-Spectrogram ADADeH 44
IV.4 Kurva Akurasi dan *Loss* Selama Pelatihan Model CNN ADADeH (46 Epoch) 54

DAFTAR TABEL

II.1 Spesifikasi perangkat pelatihan model pada Kaggle Notebook . . . 11
II.2 Spesifikasi laptop pengembangan dan pengujian 12
II.3 Ringkasan perangkat lunak yang digunakan pada ADADeH 14
II.4 Peran protokol komunikasi pada rancangan ADADeH 16

III.1 Sumber, kata kunci, dan kriteria seleksi literatur 23
III.2 Sintesis tematik hasil studi literatur 30
III.3 Keterlacakan kebutuhan pengguna terhadap literatur dan rumusan masalah 31
III.4 Kriteria penilaian alternatif solusi 31
III.5 Matriks keputusan alternatif solusi 31
III.6 Analisis sensitivitas terhadap skema bobot 32
III.7 Kebutuhan fungsional ADADeH 33
III.8 Kebutuhan nonfungsional ADADeH 34
III.9 Keterlacakan kebutuhan pengguna terhadap kebutuhan sistem . . . 35

IV.1 Pemetaan Tahap Alur Proses terhadap Kebutuhan Fungsional 39
IV.2 Elemen Keluaran ADADeH bagi Pengguna 40
IV.3 Modul Penyusun Paket `adadeh` 43
IV.4 Rincian Lapisan Model CNN ADADeH 44
IV.5 Isi Berkas Konfigurasi `meta.json` 45
IV.6 Pustaka yang Digunakan pada Implementasi ADADeH 47
IV.7 Statistik Durasi dan *Sampling Rate* Dataset ADADeH 48
IV.8 Komposisi Data Latih, Validasi, dan Uji (Tingkat Berkas dan Potongan) 49
IV.9 Parameter Pemotongan dan Ekstraksi Mel-Spectrogram 50
IV.10 Konfigurasi Pelatihan Model CNN ADADeH 53
IV.11 Hasil Evaluasi pada Data Uji – Tingkat Potongan ($n = 507$) 55
IV.12 Hasil Evaluasi pada Data Uji – Tingkat Berkas ($n = 280$, Agregasi Persamaan III.2) 55
IV.13 Kebijakan Pemeriksaan Mutu Rekaman (`QualityPolicy`) 58

IV.14 Kategori Pengujian Otomatis Aplikasi ADADeH 60

DAFTAR PERSAMAAN

II.1 Persamaan Pemetaan Frekuensi Linear ke Mel 7
II.2 Persamaan Lapisan Konvolusi dengan Aktivasi ReLU 9
II.3 Persamaan Fungsi Sigmoid 9
II.4 Persamaan Penentuan Label Prediksi 10
II.5 Persamaan *Binary Cross-Entropy* 10
II.6 Persamaan Akurasi, Presisi, *Recall*, dan F1 18
II.7 Persamaan *Equal Error Rate* 19
III.1 Persamaan Skor Terbobot Alternatif Solusi 27
III.2 Persamaan Agregasi Skor Potongan Rekaman 29

DAFTAR ALGORITMA

DAFTAR *LISTING*

IV.1 Akuisisi dataset dari Hugging Face Hub 47
IV.2 Pembangunan subset data latih, validasi, dan uji 48
IV.3 Penentuan parameter pemotongan audio 49
IV.4 Fungsi ekstraksi mel-spectrogram per potongan 50
IV.5 Lapisan SpecAugment 51
IV.6 Definisi model CNN ADADeH 51
IV.7 Normalisasi mel-spectrogram 52
IV.8 Pelatihan model CNN dengan *class weight* dan *callback* 52
IV.9 Pencarian ambang keputusan dan evaluasi model 54
IV.10 Penyimpanan model dan konfigurasi 56
A.1 Binary search code 67

DAFTAR SINGKATAN

Singkatan	Deskripsi	Pemakaian pertama kali
AI	*Artificial Intelligence*	1
CNN	*Convolutional Neural Network*	2
GPU	*Graphics Processing Unit*	14

DAFTAR SIMBOL

Notasi	Deskripsi	Pemakaian pertama kali
α	Koefisien bobot untuk *loss function*, tingkat signifikansi statistik	75
β	Koefisien bobot untuk *Binary Cross-Entropy loss*	75
λ	Koefisien bobot untuk CTC *loss*, parameter regularisasi	75
θ	Parameter model *neural network*	75
σ	Deviasi standar	75
μ	Rata-rata (*mean*)	75
ϵ	*Token blank*/kosong dalam CTC, konstanta kecil untuk stabilitas numerik	75
Δ	Delta (selisih/perubahan nilai)	75
π	*Alignment path* dalam CTC	75
π_t	*Token* pada *timestep* t dalam *alignment path*	76
∇ atau ∂	Operator gradien atau turunan parsial	75
$\sum$	Operator penjumlahan	75
$\prod$	Operator perkalian	75
$\int$	Operator integral	75
argmax	Argumen yang memaksimalkan fungsi	76
$\sim$	Terdistribusi menurut	75
$\rightarrow$	Menuju atau konvergen ke	75
$\mathcal{L}$	Fungsi *loss* (kerugian)	75
$\mathcal{L}_{total}$	Total *loss function*	75
$\mathcal{L}_{adversarial}$ atau $\mathcal{L}_{adv}$	*Adversarial loss*	75
$\mathcal{L}_{reconstruction}$	*Reconstruction loss*	75
$\mathcal{L}_{L1}$	L1 *loss* (*Mean Absolute Error*)	75
$\mathcal{L}_{L2}$	L2 *loss* (*Mean Squared Error*)	75
$\mathcal{L}_{CTC}$	CTC (*Connectionist Temporal Classification*) *loss*	75
$\mathcal{L}_{perceptual}$	*Perceptual loss*	75

$\mathcal{L}_{pixel}$	*Pixel-wise loss*	75
$\mathcal{L}_{BCE}$	*Binary Cross-Entropy loss*	75
$\mathcal{L}_{GAN}$	GAN *loss*	75
$\mathcal{L}_{cycle}$	*Cycle consistency loss*	75
G	*Generator* dalam GAN	75
D	*Discriminator* dalam GAN	75
$G(z)$	*Output* dari *generator* dengan *input* z	75
$D(x)$	*Output* dari *discriminator* dengan *input* x	75
$G_{A \to B}$	*Generator* dari domain A ke B	75
$G_{B \to A}$	*Generator* dari domain B ke A	75
$V(D, G)$	Fungsi nilai (*value function*) dalam GAN	75
x	Sampel data asli, *input* citra	75
y	Label atau *ground truth*, kondisi dalam *conditional GAN*	75
z	*Noise vector* atau representasi laten	
$\mathbb{E}$	Ekspektasi atau nilai harapan	75
p_{data}	Distribusi data asli	75
p_z	Distribusi *prior* (*noise*)	75
$p(y\|x)$	Probabilitas bersyarat *output* y *given input* x	75
$p_t(\pi_t\|x)$	Probabilitas *token* pada *timestep* t	
$\mathcal{B}$	Fungsi penciutan (*collapse function*) dalam CTC	75
$\mathcal{B}^{-1}$	Fungsi invers penciutan dalam CTC	75
T	Jumlah *timestep* atau panjang *sequence*	
$\|x - G(y)\|_1$	L1 *distance* antara *ground truth* dan *generated image*	75
$\|x - G(y)\|_2$	L2 *distance* antara *ground truth* dan *generated image*	75
$N \times N$	Ukuran *patch* dalam PatchGAN	
d	Cohen's d (*effect size*)	75
n	Jumlah sampel	75
p	*p-value* (nilai probabilitas statistik)	75

r	Koefisien korelasi	75
R^2	Koefisien determinasi	
$O(n)$	Notasi Big-O untuk kompleksitas linear	75
$O(n^2)$	Notasi Big-O untuk kompleksitas kuadratik	75

BAB I

PENDAHULUAN

I.1 Latar Belakang

Tindak kriminal pada ruang digital yang memanfaatkan *artificial intelligence* (AI) untuk memanipulasi audio/suara manusia kian meningkat dan sangat memprihatinkan. AI dimanfaatkan para pelaku kejahatan untuk membuat *deepfake audio* yang ditujukan kepada korban sebagai modus penipuan atau pemerasan. Kebanyakan korban teperdaya oleh hasil manipulasi tersebut, yang mengakibatkan kerugian pada banyak aspek, khususnya kerugian finansial.

Para pelaku kejahatan melakukan hal tersebut dengan tujuan untuk menutup identitas aslinya. Metode tersebut berpotensi menimbulkan fitnah digital, manipulasi alat bukti elektronik, hingga kesalahan identifikasi dalam proses penegakan hukum (Iman, 2026). Upaya penegakkan hukum untuk memberantas permasalahan ini memerlukan bantuan forensik digital. Forensik digital diperlukan dalam memperoleh bukti-bukti digital, termasuk bukti *deepfake audio* yang dibuat AI. Namun, hal ini hanya dapat dilaksanakan pada laboratorium forensik digital yang memfasilitasi proses perolehan serta pendeteksian bukti digital oleh para ahli.

Saat ini, laboratorium forensik digital hanya ada di Dittipidsiber Mabes Polri dan Polda Metro Jaya. Selain itu, keterbatasan fasilitas tersebut pun diiringi dengan kurangnya ahli forensik digital, khususnya ahli yang memahami AI generatif (SIP Law Firm, 2025). Keterbatasan ini menimbulkan kesulitan bagi para korban yang ingin melakukan pelaporan atau pembuktian. Korban harus merekam audio sebagai bukti sah yang selanjutnya dilimpahkan kepada institusi terkait untuk diproses.

Oleh karena itu, diperlukan suatu metode yang mempermudah korban untuk mendeteksi *deepfake audio* buatan AI. Para penulis menggagas platform bertajuk *Audio Deepfake Anomaly Detection Harness* (ADADeH) untuk memerangi manipulasi AI dengan AI (AI vs AI) yang dilatih untuk mendeteksi *deepfake audio*. Dengan demikian, pembuktian tidak perlu melalui serangkaian prosedur yang rumit.

I.2 Rumusan Masalah

Keterbatasan sarana pembuktian yang ada menimbulkan beberapa persoalan dalam menghadapi kejahatan berbasis *deepfake audio*. Proses pendeteksian saat ini bergantung pada laboratorium forensik digital dan tenaga ahli yang jumlahnya terbatas, sehingga korban sulit mendapatkan indikasi awal atas rekaman yang diterimanya. Selain itu, suara hasil AI generatif kian menyerupai suara asli manusia sehingga tidak dapat dibedakan hanya dengan pendengaran. Di sisi lain, hasil deteksi yang ada umumnya belum disajikan dalam bentuk yang mudah dipahami masyarakat awam.

Apabila persoalan-persoalan ini tidak segera ditangani, korban akan tetap kesulitan melakukan pelaporan dan pembuktian, sementara pelaku dapat terus memanfaatkan *deepfake audio* untuk menutupi identitasnya. Oleh karena itu, tugas proyek ini mengusulkan ADADeH, yaitu platform berbasis AI yang mendeteksi anomali pada rekaman audio untuk membedakan suara asli manusia dari *deepfake audio*.

Berdasarkan hal tersebut, rumusan masalah setidaknya mencakup hal-hal berikut.

1. Bagaimana mendeteksi anomali pada rekaman audio yang membedakan suara asli manusia dari *deepfake audio* hasil AI generatif?
2. Bagaimana merancang *pipeline* ADADeH, mulai dari ekstraksi fitur audio hingga klasifikasi, dan evaluasi kinerja menggunakan metrik yang sesuai?
3. Bagaimana menyajikan hasil deteksi ADADeH secara mudah sebagai indikasi awal pembuktian, tanpa bergantung pada laboratorium forensik digital?

I.3 Tujuan

Berdasarkan latar belakang dan rumusan masalah yang telah diuraikan, tujuan dari TPdK ini adalah mengembangkan ADADeH, yaitu *platform* berbasis AI untuk mendeteksi *deepfake audio* sehingga dapat memberikan indikasi awal pembuktian bagi korban tanpa bergantung pada laboratorium forensik digital. Secara rinci, tujuan tersebut dijabarkan sebagai berikut.

1. Membangun model deteksi anomali yang dapat membedakan suara asli manusia dari *deepfake audio* hasil AI generatif berdasarkan karakteristik akustik rekaman.
2. Merancang dan mengimplementasikan *pipeline* yang mencakup ekstraksi fitur audio, klasifikasi, dan evaluasi kinerja dengan metrik yang sesuai.
3. Menyajikan hasil deteksi dalam bentuk yang mudah dipahami orang awam sebagai indikasi awal pembuktian.

Tugas proyek ini dinyatakan berhasil apabila ketiga tujuan tersebut tercapai. Kriteria keberhasilan yang digunakan adalah sebagai berikut.

1. Model deteksi mencapai akurasi dan *F1-score* minimal 90% serta *Equal Error Rate* (EER) maksimal 10% pada data uji yang tidak digunakan dalam proses pelatihan.
2. *Pipeline* ADADeH dapat dijalankan secara utuh, mulai dari masukan rekaman audio hingga keluaran hasil klasifikasi, tanpa adanya intervensi manual.
3. Hasil deteksi ditampilkan dalam bentuk yang dapat dipahami oleh pengguna awam, yaitu label klasifikasi beserta tingkat keyakinan model.

I.4 Lingkup dan Batasan

Lingkup dan batasan pengembangan ADADeH adalah sebagai berikut.

1. ADADeH membantu korban kejahatan berbasis *deepfake audio* memperoleh indikasi awal atas keaslian suatu rekaman. Sistem mencakup pemrosesan rekaman audio, ekstraksi fitur, klasifikasi suara asli dan suara hasil AI generatif, serta penyajian hasil deteksi kepada pengguna.
2. Pelatihan dan pengujian model menggunakan satu *dataset* publik Kaggle berisi rekaman berbahasa Inggris. Kinerja sistem pada teknik *deepfake* lain di luar *dataset* dan pada rekaman berbahasa Indonesia belum dapat dipastikan.
3. Sistem hanya melakukan klasifikasi biner untuk membedakan suara asli dan *deepfake audio*, tanpa mengidentifikasi pelaku maupun teknik pembuatan *deepfake* yang digunakan.
4. Masukan sistem berupa rekaman audio dalam format digital dengan kualitas rekaman yang memadai. Rekaman dengan *noise* yang sangat tinggi atau durasi yang terlalu singkat dapat menurunkan akurasi deteksi.
5. Hasil deteksi tidak menggantikan pemeriksaan forensik digital oleh ahli di laboratorium forensik digital.
6. Pengembangan dibatasi hingga integrasi dan pengujian purwarupa. Penerapan dalam skala luas dan pemeliharaan sistem belum menjadi bagian dari tugas proyek ini.

I.5 Metodologi Pengembangan

Pengembangan ADADeH menggunakan model *waterfall* dari analisis kebutuhan hingga integrasi dan pengujian. Keluaran setiap tahap menjadi dasar tahap berikutnya. Hasil akhirnya adalah purwarupa yang dapat dijalankan dan didemonstrasikan. Selain itu, pendekatan *Cross-Industry Standard Process for Data Mining* (CRISP-DM) digunakan untuk memandu penyiapan data audio, pemodelan, dan evaluasi model sebelum diintegrasikan ke dalam purwarupa. Jika hasil evaluasi belum memenuhi target, data atau model disempurnakan sebelum integrasi akhir yang bersifat iteratif sampai seluruh komponen tervalidasi.

1. **Analisis kebutuhan.** Masalah *deepfake audio* dan kebutuhan pengguna ditelaah melalui latar belakang serta literatur. Kebutuhan fungsional dan nonfungsional dirumuskan, lalu sedikitnya tiga alternatif solusi dibandingkan dengan matriks keputusan.
2. **Perancangan.** Arsitektur purwarupa dan aliran data dirancang dari masukan audio, prapemrosesan, ekstraksi fitur, dan klasifikasi hingga keluaran label serta tingkat keyakinan dalam teks. Perancangan juga mencakup pemisahan data latih dan uji serta antarmuka bagi pengguna awam.
3. **Implementasi.** Rekaman berbahasa Inggris dari *Audio Deepfake Detection Dataset* di Kaggle disiapkan. Komponen pemrosesan audio, model klasifikasi suara asli dan *deepfake audio*, serta antarmuka purwarupa dibangun sesuai rancangan.
4. **Integrasi dan pengujian.** Komponen digabungkan menjadi alur otomatis dari rekaman hingga hasil klasifikasi. Pada data uji yang tidak digunakan untuk pelatihan, matriks konfusi digunakan untuk menghitung akurasi dan *F1-score*. Model ditargetkan mencapai kedua metrik tersebut minimal 90% serta *Equal Error Rate* (EER) maksimal 10%. Evaluasi pengguna menilai keterpahaman label dan tingkat keyakinan.

I.6 Sistematika Penulisan

Laporan tugas proyek ADADeH disusun dalam enam bab sebagai berikut.

1. **Bab I Pendahuluan.** Bab ini memuat latar belakang, rumusan masalah, tujuan, lingkup dan batasan, metodologi pengembangan, serta sistematika penulisan.
2. **Bab II Dasar Teori.** Bab ini membahas konsep sistem multimedia, karakteristik audio dan *deepfake audio*, dasar pemrosesan dan deteksi audio, serta kajian solusi atau penelitian terkait.
3. **Bab III Analisis Kebutuhan.** Bab ini menjelaskan kebutuhan pengguna dan sistem, sedikitnya tiga alternatif solusi, serta dasar pemilihan solusi yang dikembangkan.
4. **Bab IV Desain dan Implementasi.** Bab ini menguraikan arsitektur ADADeH, aliran pengolahan audio, rancangan antarmuka, dan implementasi purwarupa.
5. **Bab V Pengujian dan Diskusi.** Bab ini menyajikan pengujian sistem dan model, evaluasi pengguna, serta pembahasan hasil dan keterbatasannya.
6. **Bab VI Kesimpulan dan Saran.** Bab ini menyimpulkan pencapaian tujuan ADADeH dan menyampaikan saran pengembangan.

BAB II

DASAR TEORI

II.1 Data Multimedia dan Representasinya

Keberadaan data merupakan suatu hal yang krusial dalam proses pengembangan platform ADADeH. Data yang menjadi fondasi utama pengembangan ADADeH merupakan data multimedia. Pengembangan platform ini setidaknya mencakup dua data multimedia: (1) audio sebagai data utama dan (2) citra spektrogram frekuensi audio terhadap waktu yang menjadi masukan model. Data-data tersebut berkontribusi dalam mengklasifikasi audio asli dan audio buatan AI generatif (*deepfake audio*).

II.1.1 Data Audio

Menurut Winer dan Lawrence (2025), audio pada dasarnya adalah gelombang suara berbentuk pola kompresi dan ekspansi yang merambat melalui medium tertentu pada rentang frekuensi yang dapat didengar manusia. Rentang tersebut ada pada kisaran 20 Hz–20 kHz (Wikipedia Contributors, 2026). Medium rambat gelombang suara umumnya adalah udara. Pada produksi musik atau audio digital, apabila gelombang tersebut ditangkap dari udara oleh media elektronik seperti mikrofon, suara dapat direpresentasikan sebagai sinyal listrik yang dialirkan melalui kabel atau sirkuit.

Dalam TPdK ini, audio berperan sebagai data primer yang diolah dan nantinya dilatih model. Pemanfaatan audio sebagai data perlu memerhatikan beberapa hal, yaitu:

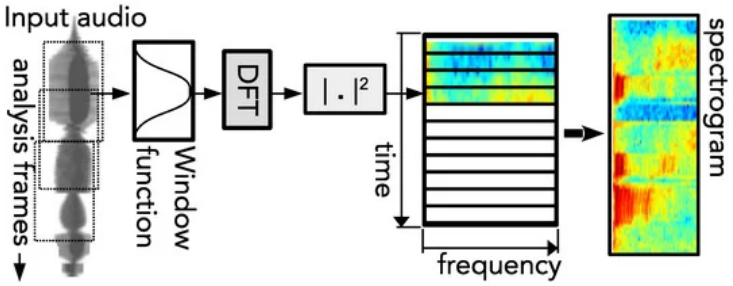
(a). *Sampling rate* (*sr*). Menurut Shannon (1949), sinyal yang tidak memuat frekuensi di atas F dapat direkonstruksi apabila dicuplik minimal $2F$ kali per detik. Audio diseragamkan pada *sr* yang sama sebelum diubah ke spektrogram.

(b). Format dan *codec*. *Dataset* yang digunakan menyimpan rekaman dalam format FLAC (*Free Lossless Audio Codec*), yaitu format kompresi audio secara *lossless* sehingga sinyal dekode identik dengan sinyal asli (van Beurden & Weaver, 2024). Hal ini berbeda dengan MP3 yang menggunakan kompresi *lossy* dan membuang sebagian informasi sinyal (Brandenburg, 1999).

(c). Jumlah kanal dan durasi. Audio perlu diseragamkan menjadi satu kanal (mono) berdurasi tetap, agar menghasilkan spektrogram berdimensi seragam.

II.1.2 Data Citra Spektrogram

Spektrogram adalah citra dua dimensi yang memuat grafik representasi gelombang suara (*waveform*) terhadap waktu. Umumnya, grafik ini menempatkan frekuensi suara pada sumbu-Y dan waktu pada sumbu-X (I. McLoughlin dkk. 2026). Spektrogram direpresentasikan sebagai matriks vektor frekuensi bertumpuk yang tiap tumpukannya mewakili magnitudo frekuensi pada bingkai waktu singkat (*analysis frame*). Pembentukkan spektrogram diilustrasikan pada Gambar II.1.



Gambar II.1 Proses Pembentukkan Spektrogram (I. McLoughlin dkk. 2026)

Setiap vektor magnitudo frekuensi merupakan hasil dari transformasi ortogonal frekuensi suara terhadap waktu seperti *Discrete Fourier Transform* (DFT), *Fast Fourier Transform* (FFT), *Discrete Cosine Transform* (DCT), dan sebagainya (I. V. McLoughlin, 2016). Proses dilanjutkan dengan penjedelaan tiap potongan audio melalui fungsi jendela (*window function*) untuk mencegah kebocoran spektral (*spectral leakage*) dan efek tepi (*edge effect*) (Gibbs, 1899). Hal ini dilakukan karena fungsi jendela biasanya meredup hingga nol atau mendekati nol di tepi setiap bingkai.

Salah satu bentuk pengembangan spektrogram adalah *mel-spectrogram*. Bentuk tersebut digunakan pada TPdK ini yang akan lebih lanjut pada Subsubbab II.1.2.1. Selain itu beberapa perlakuan khusus yang harus diterapkan pada grafik spektrogram sebelum dilatih oleh model secara umum akan dijelaskan pada Subsubbab II.1.2.2

II.1.2.1 *Mel-Spectrogram*

Representasi frekuensi terhadap waktu yang dipetakan pada skala Mel disebut *mel-spectrogram*. Skala ini pertama kali digagas oleh Stevens dkk. (1937) dengan menerapkan pemanfaatan data kenyaringan setara (*equal-loudness*) manusia untuk pemetaan frekuensi (Hz) ke skala non-linear sesuai dengan persepsi manusia. Pada skala ini, selisih nilai yang sama berkorespondensi dengan selisih tinggi nada yang terdengar sama bagi pendengar, sehingga resolusinya lebih rapat pada frekuensi rendah dan lebih renggang pada frekuensi tinggi. Secara umum, pemetaan dari frekuensi linear (f_{Hz}) ke frekuensi mel (f_{mel}) mengikuti formula komputasi di bawah:

$$f_{mel} = 2595 \log_{10} \left(1 + \frac{f_{Hz}}{700}\right) \tag{II.1}$$

Persamaan II.1 di atas menunjukkan bahwa pada $f_{Hz} \leq 700$, hubungan antara f_{Hz} dan f_{mel} akan mendekati linear. Sedangkan pada $f_{Hz} > 700$ nilai mel akan bertambah semakin lambat secara logaritmik. Frekuensi Mel yang terhitung akan menjadi tumpukkan vektor seperti halnya pada spektrogram linear. Berdasarkan keserupaan tersebut, luaran yang dihasilkan adalah suatu grafik yang disebut *mel-spectrogram*.

Karena proses pembentukannya melibatkan skala Mel yang dikembangkan dari eksperimen persepsi pendengaran manusia, *mel-spectrogram* terbukti efektif untuk berbagai tugas analisis ucapan (*speech analysis*). Keefektifan tersebut berpotensi mendukung pengembangan platform ADADeH yang fokus utamanya adalah menganalisis baik ucapan asli manusia maupun ucapan sintesis buatan AI. Selain itu, model pembelajaran mendalam atau DL (*Deep Learning*) cenderung memiliki kemampuan lebih baik dalam memproses data citra dibanding data audio mentah. Model DL seperti CNN (*Convolutional Neural Network*) merupakan salah satu model dengan kemampuan tersebut. Model ini akan dijabarkan secara rinci pada Subbab II.4.

II.1.2.2 Pemrosesan Spektrogram

Meskipun spektrogram umumnya tersaji dalam bentuk citra dua dimensi yang diproses oleh model DL, pada dasarnya spektrogram adalah grafik. Hal ini menjadi kurang tepat jika pemrosesan datanya diserupakan dengan data citra lainnya. (I. McLoughlin dkk. 2026) memaparkan bahwa ada beberapa penyesuian yang perlu dilakukan pada citra spektrogram. Beberapa perlakuan tersebut tersaji pada rincian di bawah:

1. Pemilihan Warna (Berwarna/RGB atau *Greyscale*)
    - Spektrogram pada dasarnya *greyscale* dengan nilai piksel 0 (putih)–1 (hitam). Pemberian warna pada spektrogram bisa saja dilakukan tapi hanya terbatas untuk keperluan visualisasi.
    - Memasukkan spektrogram berwarna pada model DL (seperti CNN) tidak menyebabkan kegagalan pembelajaran model.
    - Memasukkan spektrogram RGB pada model DL berimbas pada tingginya kompleksitas masukan (yaitu tiga kali lipat daripada *greyscale*). Hal ini disebabkan bahwa citra RGB memiliki 3 kanal atau *channel* warna, sedangkan greyscale hanya terdiri atas 1 kanal warna.

2. Invariansi Translasi dan Skala
    - Pada citra, objek yang digeser tetaplah objek yang sama. Hal ini berbeda dengan spektrogram yang memuat grafik frekuensi suara–waktu.
    - Pada spektrogram, pergeseran sepanjang waktu (sumbu-X) tidak mengubah isi suara, tetapi pergeseran sepanjang frekuensi (sumbu-Y) bisa membuatnya terdengar seperti suara lain. Informasi yang terkandung pada sumbu X dan Y punya makna yang berbeda.
    - Perubahan skala secara horizontal berimbas pada durasi (cepat atau lambatnya suara). Perubahan skala secara vertikal berimbas pada rentang frekuensi suara (tinggi atau rendahnya nada).
    - Hal penting lainnya adalah mempertahankan konsistensi parameter spektrogram pada tahap prapemrosesan dan pelatihan model. Hal ini bertujuan agar skala spektrogram yang diterima model sama sehingga model tidak salah membaca data.
3. Fitur Lokal
    - Korelasi lokal dan global pada audio berbeda sifatnya dengan pada citra.
    - Pola serupa pada rentang 0–50 Hz dan 16–18 kHz belum tentu bermakna, sedangkan pada citra bagian tekstur yang serupa (misalnya rumput) saling berkaitan dan penting untuk mendukung tugas klasifikasi.
    - Korelasi lokal sepanjang waktu lebih mirip selisih antar-frame video daripada titik yang berdekatan pada citra.

II.2 Sistem Multimedia

Menurut Bandung (2025), multimedia setidaknya mengacu pada penggunaan berbagai jenis media yang meliputi teks, gambar, grafik, video, audio, animasi dan sebagainya, dalam satu lingkungan terintegrasi. Lingkungan tersebut harus memiliki kapabilitas untuk memproses, menyimpan, dan mengirim media melalui jaringan komunikasi data. Lebih lanjut, terminologi sistem multimedia merujuk pada kombinasi media-media tersebut yang tersalur pada beragam aplikasi (Furht, 1994).

Selain itu, Furht (1994) juga menuturkan bahwa dalam praktiknya, sistem multimedia dapat menyimpan informasi audio dan video untuk dipakai kemudian, atau mentransmisikannya secara langsung (*live-stream*) secara *real-time*. Hal ini menyebabkan kompleksitasnya bertumpu pada seluruh komponen sistem komputer, mulai dari daya pemrosesan, penyimpanan, hingga jaringan digunakan. Dalam penerapannya pada sistem komputer, sistem multimedia kian berkembang pemanfaatannya seiring dengan berkembangnya teknologi seperti pembelajaran mendalam atau *deep learning* (DL) (Li dkk. 2021). Hal inilah yang diimplementasikan pada ADADeH.

ADADeH memposisikan sistem multimedia melalui alur pengolahan data audio. Alur ini setidaknya meliputi akuisisi, prapemrosesan, analisis model, dan penyajian model. Pada tahap akuisisi, data audio diterima dalam format FLAC (Subbab II.1.1) pada cuplikan tertentu (2,5–12,5 detik) yang konsisten. Selanjutnya data diolah pada tahap prapemrosesan dengan didekode ke plot gelombang audio (*waveform*), diseragamkan menjadi satu kanal dengan *sampling rate* dan durasi tetap, diurai dalam beberapa potongan (*chunks*), kemudian ditransformasi melalui *Short-Time Fourier transform* (STFT) dan *filter bank* mel, dikonversi ke skala logaritmik (dB), serta dinormalisasi menggunakan statistik data latih untuk menghasilkan *mel-spectrogram*. Selanjutnya *mel-spectrogram* akan dilatih pada model DL untuk tugas klasifikasi.

Pada tahap analisis model, *mel-spectrogram* $\mathbf{X} \in \mathbb{R}^{M \times T}$, dengan M jumlah *mel bin* dan T jumlah *frame* waktu, diperlakukan sebagai citra satu kanal dan diproses oleh *convolutional neural network* (CNN). Lapisan konvolusi mengekstraksi fitur lokal dengan menggeser filter berukuran $k_h \times k_w$ pada spektrogram, diikuti fungsi aktivasi ReLU (Persamaan II.2). Fitur yang dihasilkan diringkas oleh lapisan *pooling* dan diteruskan ke lapisan *fully connected* yang menghasilkan satu nilai skalar z (*logit*).

$$Z_k[i,j] = \max\left(0,\ b_k + \sum_{c=1}^{C_{\text{in}}} \sum_{m=0}^{k_h-1} \sum_{n=0}^{k_w-1} W_k[c,m,n]\, X_c[i+m,\ j+n]\right) \tag{II.2}$$

Nilai tersebut dikonversi menjadi probabilitas audio *deepfake* p menggunakan fungsi sigmoid (Persamaan II.3).

$$p = \sigma(z) = \frac{1}{1+e^{-z}} \tag{II.3}$$

Kemudian label prediksi $\hat{y}$ ditentukan dengan membandingkan p terhadap ambang τ (Persamaan II.4).

$$\hat{y} = \begin{cases} 1 & \text{jika } p \geq \tau \\ 0 & \text{jika } p < \tau \end{cases} \tag{II.4}$$

Selama pelatihan, parameter model dioptimasi dengan meminimalkan *binary cross-entropy* (Persamaan II.5).

$$\mathcal{L} = -\frac{1}{N}\sum_{i=1}^{N}\left[y_i \log p_i + (1 - y_i)\log(1 - p_i)\right] \tag{II.5}$$

Pada tahap penyajian hasil, ADADeH menampilkan label prediksi (asli atau *deepfake*) beserta skor keyakinan $c = \max(p,\, 1 - p)$ kepada pengguna.

II.3 Perangkat Keras dan Perangkat Lunak

Alur pengolahan audio pada Subbab II.2 membutuhkan perangkat untuk menjalankannya. Berbeda dengan sistem multimedia yang bergantung pada sensor atau mikrokontroler, ADADeH seluruhnya berjalan di atas komputer umum dan layanan komputasi awan (*cloud*). Oleh karena itu, subbab ini hanya membahas perangkat yang dipakai selama pengembangan purwarupa, yaitu perangkat keras pada Subbab II.3.1 dan perangkat lunak pada Subbab II.3.2.

II.3.1 Perangkat Keras

Perangkat keras pada TPdK ini dibagi berdasarkan perannya, yaitu perangkat untuk melatih model, perangkat untuk mengembangkan dan menjalankan purwarupa, serta perangkat di sisi pengguna.

II.3.1.1 Perangkat Pelatihan Model

Pelatihan model merupakan tahap yang paling berat secara komputasi. Pada tahap ini, operasi konvolusi pada Persamaan II.2 dihitung berulang kali untuk seluruh *mel-spectrogram* pada data latih, dan diulang lagi di setiap *epoch* (satu putaran penuh atas data latih). Jika hanya mengandalkan CPU (*Central Processing Unit*), proses tersebut dapat memakan waktu yang sangat lama. Hal inilah yang membuat GPU (*Graphics Processing Unit*) dibutuhkan.

GPU awalnya dirancang untuk mengolah grafis, tetapi memiliki ribuan inti kecil yang dapat mengerjakan banyak perhitungan sederhana secara bersamaan. Sifat ini cocok dengan operasi konvolusi, yang pada dasarnya adalah perkalian dan penjumlahan matriks dalam jumlah besar.

Tabel II.1 Spesifikasi perangkat pelatihan model pada Kaggle Notebook

Komponen	**Spesifikasi**
GPU	(isi, misalnya NVIDIA Tesla T4)
VRAM	(isi) GB
CPU	(isi jumlah inti dan tipenya)
RAM	(isi) GB
Penyimpanan	(isi) GB

Pelatihan model ADADeH dilakukan pada Kaggle Notebook, yaitu layanan komputasi awan yang menyediakan GPU secara gratis dengan kuota waktu tertentu tiap pekan (**kaggle2026notebooks**). Layanan ini dipilih karena *dataset* yang digunakan juga tersedia di Kaggle, sehingga data dapat langsung dipasang ke *notebook* tanpa perlu diunduh dan diunggah ulang. Spesifikasi perangkat yang disediakan tersaji pada Tabel II.1. Peran tiap komponennya adalah sebagai berikut:

(a). GPU dan VRAM (*Video Random Access Memory*). GPU mengerjakan perhitungan model, sedangkan VRAM adalah memori milik GPU yang menampung parameter model beserta sekumpulan data (*batch*) yang sedang diproses. Kapasitas VRAM membatasi banyaknya *mel-spectrogram* yang dapat diproses dalam satu *batch*.

(b). CPU. Walaupun perhitungan model dikerjakan GPU, CPU tetap berperan dalam mendekode berkas FLAC, membentuk *mel-spectrogram*, dan menyusun *batch* sebelum dikirim ke GPU.

(c). RAM (*Random Access Memory*). RAM menampung data yang sedang diolah CPU. Jika kapasitasnya kurang, proses pelatihan dapat terhenti di tengah jalan.

(d). Media penyimpanan. Media ini menyimpan *dataset* audio dan berkas model hasil pelatihan yang nantinya dimuat kembali oleh purwarupa.

II.3.1.2 Perangkat Pengembangan dan Pengujian

Laptop digunakan untuk menulis kode, menyusun laporan, dan menjalankan purwarupa ADADeH. Berbeda dengan pelatihan, proses prediksi (*inference*) jauh lebih ringan. Model cukup menghitung keluaran satu kali untuk tiap potongan audio tanpa perlu memperbarui parameternya. Oleh karena itu, purwarupa dapat dijalankan pada laptop tanpa GPU khusus. Spesifikasi laptop yang digunakan tersaji pada Tabel II.2.

II.3.1.3 Perangkat Pengguna

Dari sisi pengguna, ADADeH tidak membutuhkan perangkat khusus. Pengguna cukup memiliki perangkat yang dapat menyimpan dan mengunggah rekaman audio,

Tabel II.2 Spesifikasi laptop pengembangan dan pengujian

Komponen	**Spesifikasi**
Merek dan tipe	(isi)
Prosesor	(isi)
RAM	(isi) GB
Penyimpanan	(isi, misalnya SSD 512 GB)
Kartu grafis	(isi)
Sistem operasi	(isi)

misalnya ponsel pintar atau laptop. Meskipun begitu, ada dua komponen pada perangkat perekam yang berpengaruh langsung pada kualitas rekaman, yaitu:

(a). Mikrofon. Seperti dijelaskan pada Subbab II.1.1, mikrofon menangkap gelombang suara dari udara dan mengubahnya menjadi sinyal listrik. Mikrofon bawaan ponsel sudah memadai untuk merekam ucapan, tetapi suara bising di sekitar perekam akan ikut terekam.

(b). ADC (*Analog-to-Digital Converter*). Komponen ini mencuplik sinyal listrik dari mikrofon menjadi deretan angka pada *sampling rate* tertentu. Pada ponsel, nilainya umumnya 44,1 kHz atau 48 kHz.

Kedua komponen tersebut menentukan seberapa baik rekaman yang masuk ke sistem. Hal ini berkaitan dengan batasan pada Bab I, yaitu rekaman dengan *noise* yang sangat tinggi atau durasi yang terlalu singkat dapat menurunkan akurasi deteksi. Selain itu, rekaman dari ponsel biasanya tersimpan dalam format *lossy* seperti M4A atau OGG, sehingga perlu dikonversi terlebih dahulu ke format yang diterima sistem.

II.3.2 Perangkat Lunak

Seluruh perangkat lunak yang digunakan pada TPdK ini bersifat sumber terbuka (*open source*) atau dapat dipakai secara gratis. Perangkat lunak tersebut dikelompokkan mengikuti alur pada Subbab II.2, dan ringkasannya tersaji pada Tabel II.3.

II.3.2.1 Bahasa Pemrograman dan Lingkungan Pengembangan

(a). Python. Python menjadi bahasa pemrograman utama ADADeH. Bahasa ini dipilih karena hampir semua pustaka pemrosesan audio dan DL tersedia di dalamnya, sehingga seluruh alur dari prapemrosesan hingga antarmuka dapat ditulis dalam satu bahasa.

(b). Kaggle Notebook. Selain menyediakan perangkat keras (Subsubbab II.3.1.1), Kaggle Notebook juga menjadi tempat menulis kode pelatihan. Kode ditulis dalam sel-sel yang dapat dijalankan terpisah, dan keluarannya seperti grafik atau spektrogram langsung muncul di bawah sel. Hal ini memudahkan penulis

mencoba berbagai pengaturan tanpa menjalankan ulang seluruh program.

(c). Visual Studio Code. Editor kode ini digunakan untuk menulis kode purwarupa di laptop.

(d). Git dan GitHub. Git mencatat riwayat perubahan kode, sedangkan GitHub menyimpan kode tersebut secara daring agar seluruh anggota tim dapat bekerja pada kode yang sama.

II.3.2.2 Pustaka Pemrosesan Audio

(a). librosa. Pustaka ini menangani hampir seluruh tahap prapemrosesan (**mcfee2015librosa**). Fungsinya mencakup membaca berkas audio, menyeragamkan *sampling rate* dan jumlah kanal, menghitung STFT, menerapkan *filter bank* mel, serta mengonversi hasilnya ke skala dB.

(b). SoundFile. Pustaka ini dipakai librosa untuk mendekode berkas FLAC menjadi deretan nilai amplitudo.

(c). NumPy. *Waveform* dan *mel-spectrogram* disimpan sebagai larik (*array*) NumPy (**harris2020numpy**). Pemotongan audio menjadi *chunks* dan normalisasi dilakukan sebagai operasi pada larik tersebut.

II.3.2.3 Pustaka Pembelajaran Mendalam

(a). PyTorch. Pustaka ini digunakan untuk membangun dan melatih model CNN (**paszke2019pytorch**). PyTorch menyediakan lapisan-lapisan siap pakai seperti konvolusi, *pooling*, dan *fully connected*, serta fungsi *binary cross-entropy* pada Persamaan II.5. Selain itu, PyTorch menghitung gradien secara otomatis sehingga penulis tidak perlu menurunkan rumus pembaruan parameter secara manual.

(b). CUDA dan cuDNN. Keduanya adalah perangkat lunak dari NVIDIA yang menghubungkan PyTorch dengan GPU. Tanpa keduanya, PyTorch hanya dapat berjalan di CPU. Pada Kaggle Notebook, CUDA dan cuDNN sudah terpasang.

II.3.2.4 Pustaka Evaluasi dan Visualisasi

(a). scikit-learn. Pustaka ini digunakan untuk membagi data latih dan data uji serta menghitung matriks konfusi, akurasi, dan F1-*score* (**pedregosa2011scikit**). Kurva ROC dari pustaka ini juga dipakai untuk mencari EER, yaitu titik ketika proporsi audio *deepfake* yang lolos sebagai audio asli sama besar dengan proporsi audio asli yang salah dicap *deepfake*.

(b). Matplotlib. Pustaka ini digunakan untuk menggambar *waveform*, *mel-spectrogram*, dan grafik nilai *loss* selama pelatihan (**hunter2007matplotlib**).

Tabel II.3 Ringkasan perangkat lunak yang digunakan pada ADADeH

Perangkat Lunak	Versi	Peran
Python	(isi)	Bahasa pemrograman utama
Kaggle Notebook	–	Lingkungan pelatihan model
Visual Studio Code	(isi)	Editor kode purwarupa
Git dan GitHub	(isi)	Pengelolaan versi dan kolaborasi kode
librosa	(isi)	Prapemrosesan audio dan pembentukan *mel-spectrogram*
SoundFile	(isi)	Dekode berkas FLAC
NumPy	(isi)	Penyimpanan dan pengolahan data dalam bentuk larik
PyTorch	(isi)	Pembangunan dan pelatihan model CNN
CUDA dan cuDNN	(isi)	Penghubung PyTorch dengan GPU
scikit-learn	(isi)	Pembagian data dan penghitungan metrik evaluasi
Matplotlib	(isi)	Visualisasi data dan hasil pelatihan
Streamlit	(isi)	Antarmuka pengguna berbasis web

II.3.2.5 Perangkat Lunak Antarmuka

Antarmuka purwarupa dibangun dengan Streamlit, yaitu kerangka kerja Python untuk membuat aplikasi web sederhana tanpa perlu menulis HTML, CSS, atau JavaScript (**streamlit2026docs**). Melalui antarmuka ini, pengguna mengunggah rekaman audio dari peramban (*browser*). Sistem kemudian menjalankan seluruh alur pengolahan dan menampilkan label prediksi beserta skor keyakinannya. Cara peramban dan aplikasi saling bertukar data akan dibahas pada Subbab **??**.

Protokol komunikasi mengatur pertukaran data antarkomponen sistem, termasuk cara mengirim permintaan, menerima data, dan mengenali respons. Pada ADADeH, komunikasi tersebut menghubungkan peramban pengguna dengan aplikasi Streamlit yang menjalankan pengolahan audio. Data yang dipertukarkan berupa berkas rekaman, interaksi pengguna, dan hasil deteksi. Oleh karena itu, pembahasan protokol berkaitan dengan penyampaian rekaman secara utuh serta penyajian hasil kepada pengguna setelah pengolahan selesai.

II.3.3 Arsitektur Klien–Server dan Transportasi Data

Streamlit menggunakan arsitektur klien–server. Peramban menjadi klien yang menampilkan antarmuka, sedangkan program Python menjadi server yang menjalankan komputasi. Ketika purwarupa dijalankan secara lokal, kedua peran tersebut dapat berada pada komputer yang sama. Jika diakses melalui jaringan, perangkat pengguna dan komputer server dapat terpisah (**streamlit2026docs**). Pada rancangan ADADeH, prapemrosesan audio dan prediksi CNN berlangsung di server sehingga per-

angkat pengguna cukup mengirim rekaman dan menampilkan hasil.

Pada komunikasi berbasis TCP, *Transmission Control Protocol* menyediakan aliran bita yang andal dan berurutan. TCP menggunakan nomor urut, konfirmasi penerimaan, dan pengiriman ulang untuk menangani data yang hilang selama transmisi (**eddy2022tcp**). Sifat ini mendukung pengiriman berkas audio yang harus diterima secara lengkap sebelum didekode. Namun, keberhasilan transmisi hanya menunjukkan bahwa data dapat disampaikan; keterbacaan format dan kelayakan rekaman tetap perlu diperiksa oleh aplikasi.

II.3.4 HTTP dan HTTPS

Hypertext Transfer Protocol (HTTP) merupakan protokol pada lapisan aplikasi dengan pola permintaan dan respons. Klien mengirim permintaan ke suatu sumber daya, kemudian server mengembalikan respons yang memuat kode status dan, jika diperlukan, isi data. Sebagai contoh, metode GET digunakan untuk meminta representasi sumber daya, sedangkan POST meminta server memproses isi permintaan sesuai fungsi sumber daya tersebut (**fielding2022http**). Metode ini menjelaskan semantik komunikasi web; mekanisme unggah pada purwarupa mengikuti fasilitas yang disediakan Streamlit.

HTTPS melindungi komunikasi HTTP melalui *Transport Layer Security* (TLS). TLS menyediakan autentikasi pihak yang berkomunikasi, kerahasiaan data, dan pemeriksaan integritas untuk mendeteksi perubahan selama transmisi (**rescorla2018tls**). Perlindungan ini relevan karena rekaman yang diunggah dapat memuat suara dan percakapan pribadi. Apabila ADADeH diakses melalui jaringan publik, HTTPS perlu disediakan pada layanan yang menerima koneksi pengguna. Pengujian lokal dapat memakai HTTP, tetapi penggunaan HTTPS pada layanan publik harus dipastikan melalui konfigurasi penerapannya.

II.3.5 WebSocket dan Pengelolaan Sesi

WebSocket menyediakan kanal komunikasi dua arah yang tetap terbuka setelah proses pembentukan koneksi atau *handshake*. Pada mekanisme yang dijelaskan RFC 6455, koneksi diawali dengan permintaan HTTP untuk beralih ke protokol WebSocket. Setelah diterima, klien dan server dapat saling mengirim pesan melalui koneksi tersebut. Skema alamat `ws` digunakan untuk WebSocket, sedangkan `wss` menambahkan perlindungan TLS (**fette2011websocket**).

Streamlit menggunakan WebSocket untuk pertukaran pesan antarmuka dan pengelolaan sesi. Setiap tab peramban membentuk koneksi tersendiri yang berkaitan de-

Tabel II.4 Peran protokol komunikasi pada rancangan ADADeH

Protokol	**Peran**
TCP	Menyediakan aliran bita yang andal dan berurutan pada koneksi berbasis TCP
HTTP/HTTPS	Melayani sumber daya web dan pengiriman berkas; HTTPS melindungi komunikasi dengan TLS
WebSocket/WSS	Mendukung pertukaran pesan antarmuka dan sesi; WSS melindungi koneksi dengan TLS

ngan sesi pengguna. Pada saat yang sama, permintaan media dan unggahan berkas melibatkan HTTP (**streamlit2026docs**). Dengan demikian, pengiriman rekaman dan pembaruan tampilan merupakan bagian komunikasi yang perlu dipahami bersama. Ketika aplikasi memakai HTTPS, koneksi WebSocket juga perlu menggunakan WSS. Ketersediaan WebSocket mendukung interaksi aplikasi, sedangkan analisis ADADeH tetap dilakukan terhadap rekaman yang telah diunggah.

II.3.6 Alur Pengiriman Audio dan Penyajian Hasil

Berdasarkan arsitektur tersebut, alur komunikasi pada purwarupa ADADeH dapat dijabarkan sebagai berikut.

(a). Peramban membuka aplikasi dan membentuk sesi untuk menerima tampilan serta mengirim interaksi pengguna.
(b). Pengguna memilih rekaman melalui `st.file_uploader`. Berkas dikirim ke server, kemudian tersedia bagi program Python sebagai objek berkas yang dapat dibaca (**streamlit2026uploader**).
(c). Aplikasi memeriksa rekaman, menjalankan prapemrosesan pada Subbab II.2, dan menghasilkan prediksi melalui model yang telah dilatih.
(d). Server memperbarui antarmuka dengan label asli atau *deepfake* beserta skor model. Jika berkas gagal dibaca atau pengolahan gagal, antarmuka perlu menyampaikan penyebabnya agar pengguna dapat memperbaiki masukan.

Ringkasan peran protokol dalam alur tersebut disajikan pada Tabel II.4. HTTP dan WebSocket berperan pada pertukaran data aplikasi, sedangkan TLS melindungi koneksi ketika digunakan melalui jaringan publik.

Ukuran unggahan perlu dibatasi dan isi berkas perlu diperiksa melalui dekode audio. Penyaringan ekstensi pada `st.file_uploader` belum menjamin kesesuaian isi berkas (**streamlit2026uploader**). Pemeriksaan ini menjaga masukan sesuai dengan batasan sistem pada Bab I.

II.4 Teknologi-Teknologi Lainnya

Selain representasi data dan perangkat pada subbab sebelumnya, ADADeH memerlukan dasar deteksi *deepfake audio*, model CNN, serta metode evaluasi klasifikasi. Ketiganya menjadi landasan untuk menghubungkan rekaman masukan dengan keputusan model dan menilai kemampuan sistem secara terukur.

II.4.1 Deteksi *Deepfake Audio*

Ucapan sintetis dapat dibentuk melalui *text-to-speech* (TTS), yaitu pembangkitan ucapan dari teks, atau *voice conversion* (VC), yaitu pengubahan karakteristik suara pada ucapan sumber agar menyerupai suara sasaran. Kedua teknik tersebut menjadi bagian penting dalam kajian deteksi ucapan sintetis dan hasil konversi suara (**wang2019asvspoof**). Dalam konteks ADADeH, persoalan deteksinya dirumuskan sebagai klasifikasi biner: rekaman suara asli diberi label $y = 0$, sedangkan rekaman *deepfake* diberi label $y = 1$.

Detektor mempelajari pola yang membedakan kedua kelas dari contoh berlabel. Representasi *mel-spectrogram* pada Subsubbab II.1.2.1 memungkinkan pola energi terhadap frekuensi dan waktu menjadi masukan model. Akan tetapi, keberhasilan pada suatu kumpulan data belum memastikan keberhasilan pada teknik sintesis atau kondisi rekaman lain. Kajian ASVspoof menunjukkan perlunya evaluasi terhadap serangan yang belum dikenal dan variasi kondisi audio (**wang2019asvspoof**). Sesuai lingkup Bab I, hasil ADADeH perlu ditafsirkan dalam batas satu *dataset* berbahasa Inggris yang digunakan.

II.4.2 CNN untuk Klasifikasi Spektrogram

CNN memproses masukan melalui lapisan konvolusi untuk menghasilkan peta fitur. Pada konvolusi dua dimensi, setiap filter menggabungkan nilai dari wilayah lokal masukan dengan bobot yang dipelajari. Ukuran filter, langkah pergeseran atau *stride*, dan penambahan nilai pada tepi atau *padding* menentukan wilayah yang diproses serta dimensi keluaran (**pytorch2026conv2d**). Pada ADADeH, masukan berupa spektrogram satu kanal dengan sumbu frekuensi dan waktu sebagaimana dijelaskan pada Subsubbab II.1.2.2.

Susunan model pada Subbab II.2 menggabungkan konvolusi dan aktivasi ReLU, peringkasan melalui *pooling*, serta lapisan *fully connected* untuk menghasilkan *logit*. Konvolusi dan ReLU mengikuti Persamaan II.2, sedangkan sigmoid pada Persamaan II.3 mengubah *logit* menjadi skor p untuk kelas *deepfake*. Keputusan klasifikasi kemudian mengikuti ambang pada Persamaan II.4. Dengan susunan ini, ekstraksi

fitur dan klasifikasi dapat dipelajari dalam satu model.

PyTorch mendukung pembangunan model dan penghitungan gradien secara otomatis (**paszke2019pytorch**). Selama pelatihan, bobot diperbarui untuk mengurangi *binary cross-entropy* pada Persamaan II.5. Pada tahap prediksi, bobot yang telah dipelajari digunakan untuk menghitung keluaran rekaman baru. Pengaturan model dipilih melalui validasi sebagaimana dijelaskan pada Subbab II.4.3.

II.4.3 Pemisahan Data untuk Pelatihan dan Evaluasi

Evaluasi memerlukan pemisahan data latih, validasi, dan uji. Data latih digunakan untuk mempelajari bobot, data validasi untuk memilih pengaturan model dan ambang keputusan, serta data uji untuk menilai model setelah pilihan tersebut ditetapkan (**sklearn2026crossval**). Statistik normalisasi juga harus diperoleh dari data latih agar informasi data uji tidak memengaruhi pembentukan model (**sklearn2026pitfalls**).

Pada pengolahan rekaman menjadi beberapa potongan, seluruh potongan dari satu rekaman perlu ditempatkan pada kelompok data yang sama. Jika identitas pembicara tersedia, pemisahan berdasarkan pembicara juga perlu dipertimbangkan. Prinsip pemisahan berdasarkan kelompok mencegah contoh yang saling berkaitan muncul sekaligus pada data latih dan uji (**sklearn2026crossval**). Penerapannya pada rekaman ADADeH bertujuan mengurangi kebocoran data dan menjaga agar hasil pengujian mencerminkan kemampuan terhadap rekaman baru.

II.4.4 Metrik Evaluasi dan Interpretasi Hasil

Dengan *deepfake* sebagai kelas positif, matriks konfusi memuat empat hasil: TP untuk *deepfake* yang terdeteksi, TN untuk audio asli yang dikenali, FP untuk audio asli yang salah dianggap *deepfake*, dan FN untuk *deepfake* yang salah dianggap asli. Dari nilai tersebut, akurasi, presisi, *recall*, dan F1 dihitung menurut Persamaan II.6 (**sklearn2026metrics**).

$$\begin{aligned} \text{Akurasi} &= \frac{TP + TN}{TP + TN + FP + FN} \\ P &= \frac{TP}{TP + FP}, \qquad R = \frac{TP}{TP + FN} \\ F_1 &= \frac{2PR}{P + R} = \frac{2TP}{2TP + FP + FN} \end{aligned} \tag{II.6}$$

Akurasi menunjukkan proporsi seluruh prediksi yang benar. Presisi P menunjukkan

proporsi deteksi *deepfake* yang benar, sedangkan *recall* R menunjukkan proporsi *deepfake* yang berhasil ditemukan. F1 merangkum presisi dan *recall* melalui rata-rata harmonik sehingga melengkapi akurasi, terutama ketika jumlah contoh kedua kelas berbeda (**sklearn2026metrics**). Pada laporan evaluasi, F1 perlu dinyatakan secara konsisten untuk kelas positif tersebut.

Equal Error Rate (EER) digunakan pada tugas deteksi *deepfake* ASVspoof 2021 (**delgado2021asvspoof**). Metrik ini menyatakan tingkat kesalahan ketika proporsi audio asli yang salah ditandai sebagai *deepfake* sama dengan proporsi *deepfake* yang lolos sebagai audio asli. Untuk konvensi label ADADeH, hubungan tersebut ditulis sebagai berikut.

$$\begin{aligned} \mathrm{FPR}(\tau) &= \frac{FP(\tau)}{FP(\tau) + TN(\tau)} \\ \mathrm{FNR}(\tau) &= \frac{FN(\tau)}{FN(\tau) + TP(\tau)} \\ \mathrm{EER} &= \mathrm{FPR}(\tau_{\mathrm{EER}}) = \mathrm{FNR}(\tau_{\mathrm{EER}}) \end{aligned} \tag{II.7}$$

EER diperoleh dengan memvariasikan ambang skor. Pada data berhingga, titik kesamaan dapat diperkirakan melalui interpolasi atau pendekatan titik terdekat (**delgado2021asvspoof**). Ambang untuk menghitung EER pada data uji merupakan bagian pengukuran metrik, sedangkan ambang operasi purwarupa dipilih menggunakan data validasi. Kriteria Bab I, yaitu akurasi dan F1 minimal 90% serta EER maksimal 10%, tetap merupakan target sampai dibuktikan melalui pengujian.

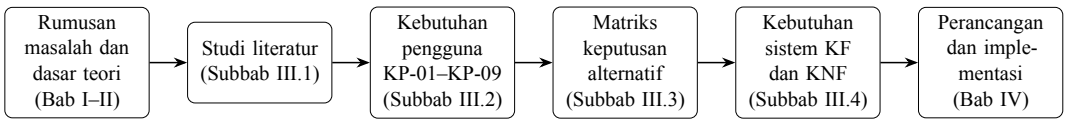
Skor keyakinan yang ditampilkan juga perlu dibedakan dari frekuensi kebenaran prediksi. Jaringan saraf dapat menghasilkan skor tinggi yang belum terkalibrasi dengan baik (**guo2017calibration**). Oleh karena itu, skor ADADeH perlu disajikan sebagai keluaran model, dan hasil klasifikasi digunakan sebagai indikasi awal sesuai batasan pada Bab I.

BAB III

ANALISIS KEBUTUHAN

Bab I telah merumuskan tiga persoalan utama ADADeH, yaitu cara mendeteksi anomali pada rekaman audio, cara merancang *pipeline* dari ekstraksi fitur hingga evaluasi, serta cara menyajikan hasil deteksi kepada masyarakat awam. Bab II kemudian menyediakan landasan teknisnya, mulai dari representasi *mel-spectrogram*, perangkat yang digunakan, hingga metrik evaluasi. Bab ini menjembatani keduanya dengan menurunkan kebutuhan yang harus dipenuhi ADADeH secara bertahap.

Alur penurunan kebutuhan diilustrasikan pada Gambar III.1. Pembahasan diawali dengan metode pengumpulan data pada Subbab III.1, dilanjutkan dengan analisis kebutuhan pengguna pada Subbab III.2. Kebutuhan pengguna tersebut menjadi dasar penilaian alternatif solusi pada Subbab III.3. Alternatif terpilih kemudian diterjemahkan menjadi kebutuhan sistem pada Subbab III.4, yang menjadi acuan perancangan pada Bab IV.



Gambar III.1 Alur Penurunan Kebutuhan ADADeH

III.1 Metode Pengumpulan Data

Data pada TPdK ini dikumpulkan melalui studi literatur. Metode ini dipilih karena tiga alasan. Pertama, pengguna sasaran ADADeH adalah korban kejahatan, sehingga pengambilan data langsung dari korban memerlukan pertimbangan etika dan perizinan di luar lingkup purwarupa pada Bab I. Kedua, karakteristik *deepfake audio* dan kondisi rekaman di dunia nyata telah dikaji secara luas melalui tantangan ASVspoof dan penelitian turunannya. Ketiga, pedoman interaksi manusia–AI serta regulasi pelindungan data tersedia sebagai dokumen yang dapat ditelaah langsung.

III.1.1 Tahapan Studi Literatur

Studi literatur dilakukan melalui enam tahap yang tersaji pada Gambar III.2. Tahap pertama adalah merumuskan pertanyaan kajian (PK) yang diturunkan dari rumusan

masalah (RM) pada Bab I, yaitu:

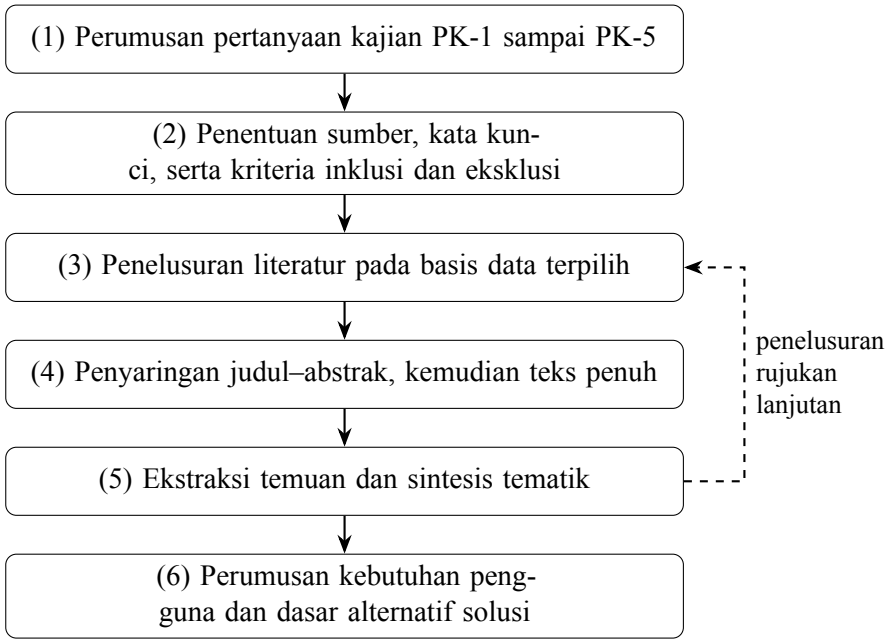
PK-1. Kondisi dan kendala apa yang dihadapi pengguna awam ketika menerima rekaman suara yang dicurigai sebagai *deepfake*? (RM-1 dan RM-3)

PK-2. Bagaimana karakteristik rekaman di dunia nyata memengaruhi kinerja detektor *deepfake audio*? (RM-1 dan RM-2)

PK-3. Bagaimana hasil sistem berbasis AI sebaiknya disajikan kepada pengguna awam? (RM-3)

PK-4. Kewajiban apa yang melekat pada pemrosesan rekaman suara sebagai data pribadi? (RM-3)

PK-5. Pendekatan deteksi apa saja yang tersedia beserta kebutuhan sumber dayanya? (RM-1 dan RM-2)



Gambar III.2 Tahapan Studi Literatur

Tahap kedua menetapkan sumber, kata kunci, serta kriteria seleksi sebagaimana tersaji pada Tabel III.1. Pada tahap ketiga dan keempat, literatur ditelusuri lalu disaring dua kali, yaitu berdasarkan judul dan abstrak, kemudian berdasarkan teks penuh. Daftar pustaka dari literatur yang lolos juga ditelusuri untuk menemukan rujukan lanjutan. Tahap kelima mengelompokkan temuan ke dalam tema, dan tahap keenam menerjemahkan tiap tema menjadi kebutuhan.

III.1.2 Hasil Studi Literatur

Literatur yang lolos penyaringan dikelompokkan ke dalam enam tema pada Tabel III.2. Tema T1 sampai T5 menjadi dasar kebutuhan pengguna pada Subbab III.2, sedangkan tema T6 menjadi dasar alternatif solusi pada Subbab III.3.

Tabel III.1 Sumber, kata kunci, dan kriteria seleksi literatur

Aspek	**Rincian**
Basis data	IEEE Xplore, ISCA Archive, ACM Digital Library, ScienceDirect, arXiv, PLOS ONE, MDPI, Google Scholar, serta JDIH untuk regulasi
Kata kunci	*audio deepfake detection*, *spoofing countermeasure*, ASVspoof, *human detection of speech deepfakes*, *voice cloning scam*, *speech privacy*, *human-AI interaction guidelines*, pelindungan data pribadi
Rentang tahun	2017–2026, kecuali rujukan dasar yang masih menjadi acuan
Kriteria inklusi	Artikel jurnal atau prosiding bertelaah sejawat, laporan resmi, dokumentasi teknis, atau regulasi; berbahasa Inggris atau Indonesia; menjawab sedikitnya satu PK
Kriteria eksklusi	Teks penuh tidak dapat diakses; tidak memuat metode atau data evaluasi; berfokus pada *deepfake* visual tanpa komponen audio; duplikasi publikasi

III.2 Analisis Kebutuhan Pengguna

Berdasarkan hasil studi literatur pada Subbab III.1.2, kebutuhan pengguna ADADeH dirumuskan pada subbab ini. Kebutuhan pengguna adalah kemampuan yang harus dimiliki sistem dari sudut pandang pengguna, belum menyangkut cara pemenuhannya secara teknis.

III.2.1 Identifikasi Pengguna

ADADeH ditujukan bagi dua kelompok pengguna berikut.

(a). **Pengguna awam.** Kelompok ini merupakan pengguna utama, yaitu korban atau calon korban kejahatan berbasis *deepfake audio*. Kelompok ini tidak memiliki latar belakang pemrosesan sinyal maupun forensik digital, sehingga membutuhkan keluaran yang langsung dapat dipahami.

(b). **Pendamping pelaporan.** Kelompok ini mencakup pihak yang membantu korban menyiapkan laporan, misalnya keluarga, pendamping hukum, atau petugas penerima aduan. Kelompok ini membutuhkan keluaran yang dapat disimpan atau dilampirkan sebagai indikasi awal, dengan kesadaran bahwa hasil tersebut bukan pengganti pemeriksaan ahli.

III.2.2 Rumusan Kebutuhan Pengguna

Kebutuhan pengguna diberi kode KP-01 sampai KP-09 dan diuraikan sebagai berikut.

KP-01. **Pengguna membutuhkan sarana pemeriksaan keaslian rekaman yang dapat diakses secara mandiri.**
Laporan **mcafee2023impostor<empty citation>** terhadap lebih dari 7.000

responden di tujuh negara menunjukkan bahwa satu dari empat responden pernah mengalami atau mengenal korban penipuan kloning suara, dan sebagian besar korban mengalami kerugian finansial. Di sisi lain, **mai2023warning<empty citation>** menemukan bahwa pendengar hanya mampu mengenali sekitar 73% sampel *deepfake*, dan pemberian contoh sebelumnya hanya sedikit meningkatkan kemampuan tersebut. Karena pemeriksaan melalui laboratorium forensik digital terbatas (SIP Law Firm, 2025), pengguna membutuhkan sarana deteksi otomatis yang dapat dipakai tanpa melalui prosedur kelembagaan terlebih dahulu.

KP-02. **Pengguna membutuhkan sistem yang menerima rekaman dari perangkat sehari-hari.**
Rekaman pengguna umumnya berasal dari ponsel dan tersimpan dalam format *lossy* seperti M4A atau OGG (Subsubbab II.3.1.3). Tugas *deepfake* pada ASVspoof 2021 secara khusus menyoroti audio terkompresi, dan hasilnya menunjukkan bahwa detektor masih kurang mampu menggeneralisasi lintas sumber data (**liu2023asvspoof2021**). Sistem karena itu harus menerima berkas tersebut dan menyeragamkannya sendiri, tanpa menuntut pengguna melakukan konversi format secara manual.

KP-03. **Pengguna membutuhkan keluaran berupa kesimpulan yang langsung dapat dipahami.**
Pedoman interaksi manusia–AI menganjurkan agar sistem menjelaskan apa yang dapat dilakukannya serta seberapa baik kinerjanya (**amershi2019guidelines**). Bagi pengguna awam, hal ini berarti pernyataan "terindikasi asli" atau "terindikasi *deepfake*" beserta tingkat keyakinannya, bukan nilai *logit* atau grafik spektrum. Kebutuhan ini sejalan dengan penyajian skor keyakinan $c = \max(p,\, 1 - p)$ pada Subbab II.2.

KP-04. **Pengguna membutuhkan keterangan mengenai keterbatasan hasil.**
Keluaran ADADeH berkaitan dengan kepentingan hukum (Iman, 2026). Selain itu, **muller2022generalize<empty citation>** menunjukkan bahwa kinerja detektor yang baik pada data tolok ukur dapat turun tajam pada rekaman dunia nyata, sedangkan skor jaringan saraf belum tentu terkalibrasi (**guo2017calibration**). Sistem karena itu harus menegaskan bahwa hasil merupakan indikasi awal, model dilatih pada satu *dataset* berbahasa Inggris, dan hasil tidak menggantikan pemeriksaan forensik digital oleh ahli.

KP-05. **Pengguna membutuhkan hasil dalam waktu yang singkat.**
Penipuan kloning suara memanfaatkan kedekatan emosional dan rasa mendesak agar korban segera bertindak (**mcafee2023impostor**). Pemeriksaan karena itu kerap dilakukan dalam situasi tertekan. Menurut **nielsen1993response<empty citation>**,

sekitar 10 detik adalah batas pengguna dapat mempertahankan perhatian saat menunggu, dan penundaan yang lebih lama memerlukan indikator kemajuan.

KP-06. **Pengguna membutuhkan sistem yang dapat memeriksa rekaman berdurasi panjang.**

Rekaman percakapan dapat berlangsung beberapa menit, sedangkan model bekerja pada potongan berdurasi tetap (Subbab II.2). **muller2022generalize<empty citation>** juga menemukan bahwa masukan sepanjang 4 detik belum memberikan kinerja maksimal dibanding masukan yang lebih panjang. Pengguna karena itu membutuhkan satu kesimpulan menyeluruh atas seluruh rekaman yang diunggah, bukan daftar hasil per potongan.

KP-07. **Pengguna membutuhkan peringatan ketika mutu rekaman tidak memadai.**

Evaluasi detektor pada audio berderau masih terbatas (**almutairi2022review**). **muller2021silence<empty citation>** juga menemukan bahwa durasi keheningan pada ASVspoof 2019 berkorelasi dengan label, sehingga model dapat "belajar" dari keheningan alih-alih dari ciri ucapan. Rekaman yang terlalu singkat, didominasi keheningan, atau mengandung derau tinggi karena itu berisiko menghasilkan keputusan yang kurang dapat dipercaya. Pengguna perlu diberi tahu hal tersebut agar tidak menyimpulkan secara keliru.

KP-08. **Pengguna membutuhkan jaminan kerahasiaan atas rekaman yang diunggah.**

Rekaman ucapan merupakan sumber data pribadi yang kaya, mulai dari informasi kesehatan hingga ciri biometrik penutur (**nautsch2019privacy**). Undang-Undang Nomor 27 Tahun 2022 menggolongkan data biometrik sebagai data pribadi yang bersifat spesifik dan mewajibkan pemrosesan yang terbatas, spesifik, dan sesuai tujuannya (**uupdp2022**). Pengguna membutuhkan kepastian bahwa berkas tidak disebarluaskan dan tidak disimpan melebihi keperluan pemrosesan.

KP-09. **Pendamping pelaporan membutuhkan ringkasan hasil yang dapat disimpan.**

Deepfake audio berpotensi dipakai untuk memanipulasi alat bukti elektronik (Iman, 2026), sementara pemeriksaan ahli memerlukan waktu (SIP Law Firm, 2025). Pendamping membutuhkan ringkasan yang mencatat identitas berkas yang diperiksa, waktu pemeriksaan, dan hasilnya, sehingga dapat dilampirkan sebagai indikasi awal ketika rekaman diserahkan kepada pihak berwenang.

III.2.3 Keterlacakan Kebutuhan

Agar setiap kebutuhan dapat ditelusuri asalnya, Tabel III.3 memetakan kebutuhan pengguna terhadap tema literatur pada Tabel III.2 dan rumusan masalah pada Bab I. Pemetaan ini sekaligus memastikan tidak ada kebutuhan yang dirumuskan tanpa dasar.

Kesembilan kebutuhan tersebut menjadi dasar penilaian alternatif solusi pada subbab berikutnya. Alternatif terpilih kemudian diterjemahkan menjadi kebutuhan sistem yang bersifat teknis.

III.3 Pemilihan Alternatif Solusi

Seluruh alternatif pada subbab ini memakai platform yang sama, yaitu aplikasi web Streamlit yang memproses rekaman di sisi server (Subbab II.3.3). Akibatnya, kebutuhan yang berkaitan dengan antarmuka dan privasi (KP-03, KP-04, KP-08, dan KP-09) dapat dipenuhi setara oleh semua alternatif. Perbedaan antaralternatif terletak pada pendekatan deteksinya, sehingga penilaian difokuskan pada aspek tersebut.

III.3.1 Alternatif Solusi

Berdasarkan tema T6 pada Tabel III.2, empat alternatif pendekatan deteksi dipertimbangkan.

(A1). **Fitur *cepstral* dengan pengklasifikasi klasik.** Rekaman diringkas menjadi fitur MFCC, LFCC, atau CQCC, kemudian diklasifikasi menggunakan GMM atau SVM. CQCC dengan GMM terbukti efektif sebagai *countermeasure*, tetapi konfigurasinya sensitif terhadap jenis serangan dan skenario penggunaan (**todisco2017cqcc**). Pendekatan ini sangat ringan, tetapi daya pembedanya terbatas pada ciri yang dirancang secara manual.

(B2). **CNN pada *mel-spectrogram*.** *Mel-spectrogram* diperlakukan sebagai citra satu kanal dan diklasifikasi oleh CNN, sebagaimana dijelaskan pada Subbab II.2 dan II.4.2. Sistem berbasis CNN ringan (LCNN) pada fitur spektral mencapai EER 1,86% pada skenario *logical access* ASVspoof 2019 (**lavrentyeva2019stc**). Model berukuran kecil hingga menengah dan dapat dijalankan pada CPU.

(C3). **Model *end-to-end* pada *waveform* mentah.** Model seperti RawNet2 (**tak2021rawnet2**) dan AASIST (**jung2022aasist**) menerima deretan sampel audio secara langsung sehingga fitur dipelajari sepenuhnya oleh jaringan. AASIST memodelkan artefak spektral dan temporal melalui *graph attention*, dan varian ringannya hanya memiliki sekitar 85 ribu parameter. Namun, pemrosesan pada resolusi sampel penuh dan arsitektur yang lebih kompleks menambah beban pelatihan serta penyesuaian.

(D4). **Model *self-supervised* pralatih.** *Front-end* wav2vec 2.0 XLS-R yang telah dilatih pada data ucapan berskala besar disetel ulang (*fine-tuning*) bersama *back-end* AASIST dan augmentasi data. Pendekatan ini melaporkan EER terendah pada basis data ASVspoof 2021 *logical access* dan *deepfake* saat dipublikasikan (**tak2022wav2vec**). Kekurangannya, *front-end* tersebut berukuran sekitar 300 juta parameter sehingga berat untuk dilatih dan dijalankan tanpa GPU.

III.3.2 Kriteria Penilaian

Kriteria penilaian beserta bobotnya tersaji pada Tabel III.4. Bobot terbesar diberikan pada kinerja deteksi karena kriteria tersebut merupakan syarat keberhasilan utama pada Bab I. Kebutuhan sumber daya pelatihan dan waktu inferensi diberi bobot berikutnya karena keduanya merupakan batasan nyata pada Subbab II.3.1. Kemudahan implementasi serta ketahanan terhadap kondisi rekaman melengkapi penilaian.

Setiap alternatif diberi skor s_{ij} pada skala 1 (sangat kurang) sampai 5 (sangat baik) untuk setiap kriteria. Skor akhir alternatif j dihitung sebagai jumlah skor terbobot pada Persamaan III.1, dengan w_i bobot kriteria ke-i dan $\sum_i w_i = 1$.

$$S_j = \sum_{i=1}^{5} w_i \, s_{ij} \tag{III.1}$$

III.3.3 Matriks Keputusan

Hasil penilaian tersaji pada Tabel III.5. Dasar pemberian skor untuk setiap alternatif adalah sebagai berikut.

(a). **A1** unggul pada K2 dan K3 karena tidak memerlukan GPU. Namun, skornya rendah pada K1 dan K5 karena sistem berbasis DL secara konsisten melampaui pendekatan klasik pada tantangan ASVspoof (**lavrentyeva2019stc**), sedangkan konfigurasi CQCC sensitif terhadap jenis serangan (**todisco2017cqcc**).

(b). **A2** memiliki kinerja yang kompetitif (**lavrentyeva2019stc**) dengan model yang cukup kecil untuk dilatih dalam kuota Kaggle dan dijalankan pada CPU. Pustaka librosa dan PyTorch (Subbab II.3.2) mendukung seluruh tahapnya. Skor K5 bernilai sedang karena **muller2022generalize<empty citation>** melaporkan bahwa fitur *mel-spectrogram* rata-rata kalah dibanding spektrogram logaritmik atau CQT.

(c). **A3** berpotensi menangkap artefak yang tidak tampak pada fitur rancangan ma-

nual (**tak2021rawnet2**), tetapi pemrosesan pada resolusi sampel penuh serta arsitektur *graph attention* (**jung2022aasist**) menambah beban pelatihan, inferensi, dan penyesuaian.

(d). **A4** memiliki kinerja dan ketahanan tertinggi (**tak2022wav2vec**), tetapi ukuran *front-end* yang besar membuatnya paling berat pada K2, K3, dan K4.

Untuk memeriksa apakah keputusan bergantung pada pilihan bobot, perhitungan diulang dengan dua skema bobot lain. Hasilnya tersaji pada Tabel III.6. Pada skema kedua, bobot K1 dinaikkan menjadi 0,40 dan bobot kriteria lain diturunkan secara proporsional. Pada skema ketiga, seluruh kriteria diberi bobot sama.

III.3.4 Solusi Terpilih

Alternatif A2, yaitu CNN pada *mel-spectrogram*, memperoleh skor tertinggi pada ketiga skema bobot. Dengan demikian, keputusan tidak bergantung pada satu pilihan bobot tertentu. A2 dipilih karena menyeimbangkan kinerja deteksi dengan keterbatasan perangkat pada Subbab II.3.1, dan sejalan dengan landasan teori pada Bab II.

Pilihan ini membawa satu risiko yang perlu dicatat, yaitu temuan **muller2022generalize<empty citation>** bahwa fitur *mel-spectrogram* dapat kalah dibanding spektrogram logaritmik. Sesuai pendekatan CRISP-DM pada Bab I, apabila evaluasi belum memenuhi target, representasi masukan dapat diganti dengan spektrogram logaritmik tanpa mengubah arsitektur sistem secara keseluruhan. Alternatif A4 tetap menjadi arah pengembangan lanjutan apabila sumber daya komputasi yang lebih besar tersedia.

III.4 Analisis Kebutuhan Sistem

Kebutuhan pengguna pada Subbab III.2 dan solusi terpilih pada Subbab III.3.4 diterjemahkan menjadi kebutuhan sistem. Kebutuhan sistem dibagi menjadi kebutuhan fungsional (KF), yaitu apa yang harus dilakukan sistem, dan kebutuhan nonfungsional (KNF), yaitu mutu yang harus dipenuhi sistem saat melakukannya.

III.4.1 Kebutuhan Fungsional

Kebutuhan fungsional ADADeH tersaji pada Tabel III.7. KF-02 sampai KF-08 dijalankan berurutan secara otomatis setelah pengguna mengunggah berkas, sehingga memenuhi kriteria keberhasilan kedua pada Bab I, yaitu *pipeline* yang berjalan tanpa intervensi manual.

Agregasi pada KF-07 dilakukan dengan merata-ratakan skor seluruh potongan, sebagaimana tersaji pada Persamaan III.2. Skor rekaman $\bar{p}$ kemudian menggantikan p pada Persamaan II.4 untuk menentukan label, dan skor keyakinan c dihitung dari

nilai yang sama. Aturan agregasi lain, misalnya nilai maksimum atau pemungutan suara mayoritas, dapat dibandingkan pada data validasi sebelum ditetapkan.

$$\bar{p} = \frac{1}{K} \sum_{k=1}^{K} p_k, \qquad c = \max\left(\bar{p},\, 1 - \bar{p}\right) \tag{III.2}$$

III.4.2 Kebutuhan Nonfungsional

Kebutuhan nonfungsional ADADeH tersaji pada Tabel III.8. Setiap kebutuhan disertai cara verifikasinya agar dapat diuji pada Bab V.

KNF-08 diturunkan dari Subsubbab II.1.2.2, yang menegaskan bahwa parameter spektrogram pada tahap pelatihan dan prediksi harus sama agar model tidak salah membaca data. Batas pada KNF-02 dan KNF-03 bersifat awal dan dapat disesuaikan berdasarkan hasil pengujian purwarupa.

III.4.3 Keterlacakan Kebutuhan Pengguna terhadap Kebutuhan Sistem

Tabel III.9 memetakan setiap kebutuhan pengguna terhadap kebutuhan sistem yang memenuhinya. Seluruh kebutuhan pengguna terpenuhi oleh sedikitnya satu kebutuhan sistem, sedangkan KF-05, KF-06, dan KNF-08 merupakan konsekuensi teknis dari pemilihan alternatif A2.

Dengan terumuskannya kebutuhan fungsional dan nonfungsional, ADADeH memiliki acuan yang terukur untuk tahap perancangan. Bab IV akan menerjemahkan kebutuhan tersebut menjadi arsitektur sistem, aliran pengolahan audio, dan rancangan antarmuka purwarupa.

Tabel III.2 Sintesis tematik hasil studi literatur

Tema	Literatur	Temuan Pokok	Implikasi
T1	(**mcafee2023imposter**; **mai2023warning**; Iman, 2026; SIP Law Firm, 2025)	Satu dari empat responden pernah mengalami atau mengenal korban penipuan kloning suara. Pendengar hanya mengenali 73% sampel *deepfake*, dan pelatihan hanya sedikit membantu. Sarana forensik digital terbatas.	Alat deteksi otomatis yang dapat diakses mandiri dan cepat (KP-01, KP-05)
T2	(**liu2023asvspoof2021**; **muller2022generalize**; **almutairi2022review**; **yi2023survey**)	Kompresi dan transmisi mengubah sinyal. Detektor kurang mampu menggeneralisasi lintas sumber data, dan kinerjanya turun tajam pada rekaman dunia nyata. Evaluasi pada audio berderau dan berbahasa non-Inggris masih terbatas.	Penerimaan format sehari-hari, peringatan mutu, dan pernyataan batasan (KP-02, KP-04, KP-07)
T3	(**muller2021silence**; **muller2022generalize**)	Durasi keheningan berkorelasi dengan label pada ASVspoof 2019. Masukan 4 detik belum memberi kinerja maksimal dibanding masukan yang lebih panjang.	Penanganan keheningan yang konsisten dan analisis seluruh rekaman (KP-06, KP-07)
T4	(**amershi2019guidelines**; **guo2017calibration**; **nielsen1993response**)	Sistem AI perlu menjelaskan apa yang dapat dilakukan dan seberapa baik kinerjanya. Skor jaringan saraf belum tentu terkalibrasi. Sekitar 10 detik adalah batas perhatian pengguna saat menunggu.	Label sederhana, skor dengan keterangan, dan respons singkat (KP-03, KP-04, KP-05)
T5	(**nautsch2019privacy**; **uupdp2022**)	Rekaman ucapan memuat data pribadi yang sensitif. Data biometrik tergolong data pribadi spesifik.	Rekaman tidak disimpan maupun disebarluaskan (KP-08)
T6	(**todisco2017cqcc**; **lavrentyeva2019stc**; **tak2021rawnet2**; **jung2022aasist**; **tak2022wav2vec**)	Pendekatan deteksi berkisar dari fitur *cepstral* dengan GMM, CNN pada fitur spektral, model *end-to-end* pada *waveform* mentah, hingga model *self-supervised* pralatih.	Dasar empat alternatif solusi (Subbab III.3)

Tabel III.3 Keterlacakan kebutuhan pengguna terhadap literatur dan rumusan masalah

Kode	Tema	Literatur Pendukung	RM
KP-01	T1	(**mcafee2023impostor**; **mai2023warning**; SIP Law Firm, 2025)	3
KP-02	T2	(**liu2023asvspoof2021**)	2
KP-03	T4	(**amershi2019guidelines**)	3
KP-04	T2, T4	(**muller2022generalize**; **guo2017calibration**; Iman, 2026)	3
KP-05	T1, T4	(**mcafee2023impostor**; **nielsen1993response**)	2
KP-06	T3	(**muller2022generalize**)	2
KP-07	T2, T3	(**almutairi2022review**; **muller2021silence**)	1
KP-08	T5	(**nautsch2019privacy**; **uupdp2022**)	3
KP-09	T1	(Iman, 2026; SIP Law Firm, 2025)	3

Tabel III.4 Kriteria penilaian alternatif solusi

Kode	Kriteria	Keterangan	Bobot
K1	Kinerja deteksi	Peluang mencapai akurasi dan F1 minimal 90% serta EER maksimal 10% (Bab I; KP-04)	0,30
K2	Sumber daya pelatihan	Kesesuaian dengan GPU dan kuota pekanan Kaggle Notebook (Subsubbab II.3.1.1)	0,20
K3	Waktu inferensi	Kecepatan prediksi pada laptop tanpa GPU khusus (Subsubbab II.3.1.2; KP-05)	0,20
K4	Kemudahan implementasi	Ketersediaan pustaka dan kemudahan dipahami, di-*debug*, serta disesuaikan dalam jangka waktu TPdK	0,15
K5	Ketahanan kondisi rekaman	Kemampuan menghadapi kompresi, derau, dan variasi rekaman (KP-02, KP-07)	0,15

Tabel III.5 Matriks keputusan alternatif solusi

Kriteria	Bobot	A1	A2	A3	A4
K1 Kinerja deteksi	0,30	2	4	4	5
K2 Sumber daya pelatihan	0,20	5	4	3	2
K3 Waktu inferensi	0,20	5	4	3	1
K4 Kemudahan implementasi	0,15	4	4	3	2
K5 Ketahanan kondisi rekaman	0,15	2	3	3	5
Skor terbobot S_j		3,50	**3,85**	3,30	3,15

Tabel III.6 Analisis sensitivitas terhadap skema bobot

Skema Bobot	**A1**	**A2**	**A3**	**A4**
Bobot utama (Tabel III.4)	3,50	**3,85**	3,30	3,15
K1 dinaikkan menjadi 0,40	3,29	**3,87**	3,40	3,41
Bobot sama (0,20 per kriteria)	3,60	**3,80**	3,20	3,00

Tabel III.7 Kebutuhan fungsional ADADeH

Kode	Kebutuhan Fungsional	Dasar
KF-01	Sistem menyediakan halaman unggah rekaman melalui peramban yang menerima berkas WAV, FLAC, MP3, M4A, dan OGG.	KP-01, KP-02
KF-02	Sistem memeriksa ukuran dan durasi berkas terhadap batas pada KNF-03, lalu mendekode isinya untuk memastikan berkas benar-benar audio. Berkas yang tidak lolos ditolak dengan pesan penyebab.	KP-02, KP-07
KF-03	Sistem menyeragamkan rekaman menjadi satu kanal pada *sampling rate* data latih, serta menangani keheningan di awal dan akhir rekaman dengan aturan yang sama dengan tahap pelatihan.	KP-02, KP-07
KF-04	Sistem memotong rekaman menjadi K potongan berdurasi tetap sesuai masukan model. Sisa potongan yang lebih pendek ditangani dengan aturan yang sama dengan tahap pelatihan.	KP-06
KF-05	Sistem membentuk *mel-spectrogram* dalam skala dB untuk setiap potongan dan menormalisasinya dengan statistik data latih.	A2
KF-06	Sistem menghitung skor *deepfake* p_k untuk setiap potongan menggunakan model CNN terlatih.	A2
KF-07	Sistem menggabungkan skor seluruh potongan menjadi satu skor rekaman (Persamaan III.2) dan menentukan label dengan ambang τ dari data validasi.	KP-06
KF-08	Sistem menampilkan label "Terindikasi asli" atau "Terindikasi *deepfake*" beserta skor keyakinan dalam persen dan penjelasan singkat.	KP-03
KF-09	Sistem selalu menampilkan keterangan batasan hasil bersama label, yaitu sifatnya sebagai indikasi awal dan cakupan *dataset* pelatihan.	KP-04
KF-10	Sistem menampilkan peringatan mutu apabila rekaman terlalu singkat, didominasi keheningan, terpotong (*clipping*), atau berlevel sinyal sangat rendah.	KP-07
KF-11	Sistem menyediakan ringkasan hasil yang dapat diunduh, memuat nama berkas, durasi, waktu pemeriksaan, nilai *hash* SHA-256 berkas, label, skor, versi model, dan keterangan batasan.	KP-09
KF-12	Sistem menghapus berkas unggahan dan data antara setelah pemrosesan selesai atau sesi berakhir.	KP-08

Tabel III.8 Kebutuhan nonfungsional ADADeH

Kode	Aspek	Kebutuhan	Cara Verifikasi	Dasar
KNF-01	Kinerja deteksi	Akurasi dan F1 kelas *deepfake* minimal 90% serta EER maksimal 10% pada data uji terpisah (Subbab II.4.3).	Matriks konfusi dan kurva ROC pada data uji.	Bab I, KP-04
KNF-02	Waktu respons	Hasil untuk rekaman hingga 60 detik tersaji paling lama 10 detik setelah unggahan selesai pada laptop Tabel II.2. Rekaman yang lebih panjang disertai indikator kemajuan.	Pengukuran waktu rata-rata dari beberapa kali uji.	KP-05
KNF-03	Batas masukan	Durasi minimum satu potongan model, durasi maksimum 10 menit, dan ukuran berkas maksimum 100 MB.	Uji berkas di bawah dan di atas setiap batas.	KP-06, KP-07
KNF-04	Keamanan transmisi	Layanan yang diakses melalui jaringan publik menggunakan HTTPS dan WSS (Subbab II.3.4 dan II.3.5).	Pemeriksaan konfigurasi dan sertifikat TLS.	KP-08
KNF-05	Privasi	Rekaman tidak disimpan secara permanen, tidak dikirim ke pihak ketiga, dan isinya tidak dicatat pada *log*.	Inspeksi direktori sementara dan *log* setelah pengujian.	KP-08
KNF-06	Keterpahaman	Antarmuka berbahasa Indonesia dengan istilah awam, dan hasil diperoleh dalam paling banyak tiga langkah: membuka, mengunggah, dan membaca hasil.	Evaluasi pengguna terhadap keterpahaman label dan skor.	KP-01, KP-03
KNF-07	Kompatibilitas	Dapat diakses melalui peramban modern pada laptop maupun ponsel tanpa instalasi tambahan.	Uji pada peramban desktop dan ponsel.	KP-01, KP-02
KNF-08	Konsistensi prapemrosesan	Parameter prapemrosesan, statistik normalisasi, dan ambang τ disimpan	Skor berkas yang sama pada *notebook* pelatihan dan purwarupa bernilai	A2

Tabel III.9 Keterlacakan kebutuhan pengguna terhadap kebutuhan sistem

Kebutuhan Pengguna	**Kebutuhan Sistem**
KP-01	KF-01, KNF-06, KNF-07
KP-02	KF-01, KF-02, KF-03, KNF-07
KP-03	KF-08, KNF-06
KP-04	KF-09, KNF-01
KP-05	KNF-02
KP-06	KF-04, KF-07, KNF-03
KP-07	KF-02, KF-03, KF-10, KNF-03
KP-08	KF-12, KNF-04, KNF-05
KP-09	KF-11

BAB IV

Desain dan Implementasi

Bab III telah menurunkan kebutuhan fungsional (KF-01 sampai KF-12) dan kebutuhan nonfungsional (KNF-01 sampai KNF-08) ADADeH, serta menetapkan CNN pada mel-spectrogram (alternatif A2) sebagai pendekatan deteksi yang dipilih melalui matriks keputusan pada Subbab III.3. Bab ini menerjemahkan kebutuhan dan pilihan tersebut menjadi rancangan yang konkret dan menjabarkan purwarupa yang telah diimplementasikan secara nyata sebagai aplikasi web ADADeH berbasis Streamlit. Subbab IV.1 menguraikan desain sistem secara keseluruhan, yaitu bagaimana kebutuhan diwujudkan menjadi alur kerja dan keluaran yang diterima pengguna. Subbab IV.2 menjabarkan arsitektur sistem, mulai dari pembagian peran klien dan server, susunan paket perangkat lunak `adadeh` yang menjalankan setiap tahap pemrosesan, hingga susunan lapisan model klasifikasi. Subbab IV.3 menyajikan implementasi purwarupa secara lengkap, yaitu (a) pengembangan model pada notebook: akuisisi data, pra-pemrosesan audio, pembangunan model CNN, pelatihan, dan evaluasi awal; serta (b) pembangunan aplikasi: validasi dan dekode audio yang diperkuat dari sisi keamanan, pemuatan model dengan verifikasi integritas, pemeriksaan mutu rekaman, orkestrasi layanan dan antarmuka pengguna, serta evaluasi purwarupa yang mereproduksi baseline notebook sekaligus melengkapi metrik Equal Error Rate (EER) yang sebelumnya belum dihitung. Seluruh uraian mengacu langsung pada notebook pengembangan ADADeH dan kode sumber paket `adadeh` pada aplikasi yang telah dibangun.

IV.1 Desain Sistem

IV.1.1 Prinsip Desain

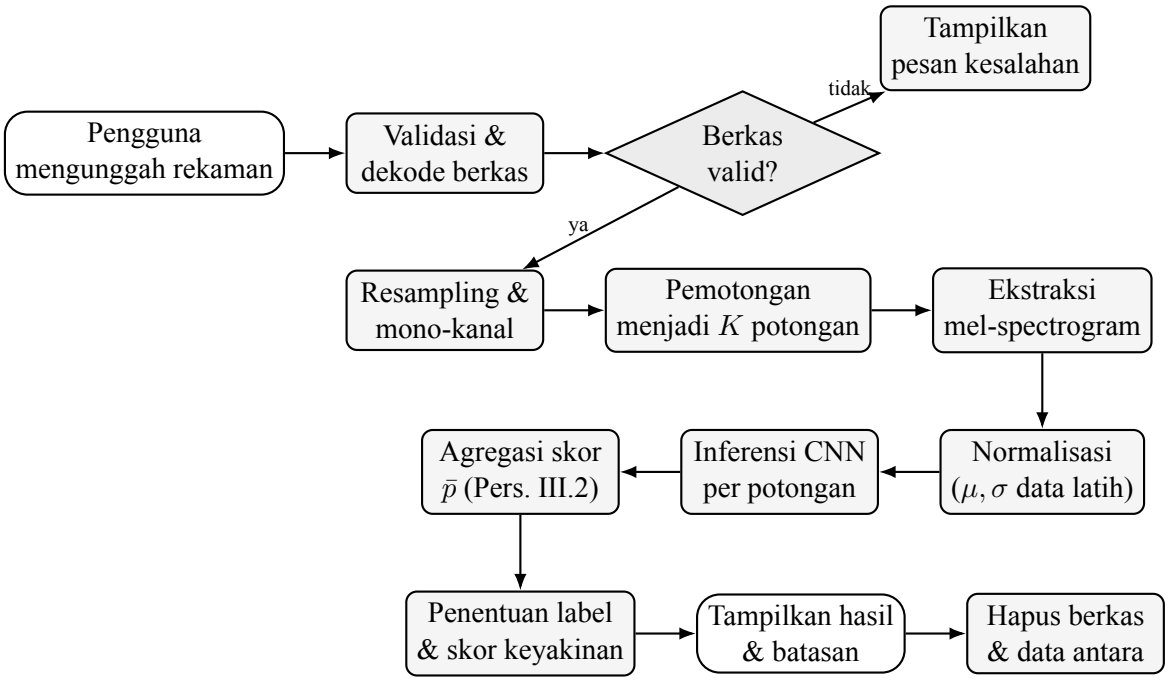
Tiga prinsip mendasari desain ADADeH, yang masing-masing diturunkan langsung dari kebutuhan sistem pada Bab III.

1. **Pipeline otomatis end-to-end.** Sejak rekaman diunggah hingga label dan skor keyakinan ditampilkan, seluruh tahap (KF-02 sampai KF-08) berjalan berurutan tanpa intervensi manual, memenuhi kriteria keberhasilan kedua pada Bab I.

2. **Konsistensi parameter latih dan infer.** Parameter pra-pemrosesan (*sampling rate*, durasi potongan, jumlah *mel bin*) serta statistik normalisasi dan ambang keputusan yang digunakan saat prediksi harus identik dengan yang digunakan saat pelatihan. Prinsip ini menjawab KNF-08 dan konsekuensi teknis yang telah ditekankan pada Subsubbab II.1.2.2, yaitu bahwa model akan salah membaca data apabila skala masukannya berubah.
3. **Pemisahan tanggung jawab antarmuka dan pemrosesan.** Mengikuti arsitektur klien–server pada Subbab II.3.3, peramban pengguna hanya bertanggung jawab menerima masukan dan menampilkan keluaran, sedangkan seluruh pemrosesan audio dan inferensi model berjalan di sisi server. Pemisahan ini memudahkan pengguna awam (KP-01, KP-03) dan menjaga agar perangkat pengguna tidak perlu memenuhi spesifikasi komputasi tertentu (Subsubbab II.3.1.3).

IV.1.2 Alur Proses Sistem

Alur proses ADADeH terdiri atas delapan tahap yang mengalir dari masukan rekaman hingga keluaran hasil deteksi. Gambar IV.1 mengilustrasikan alur tersebut, sedangkan Tabel IV.1 memetakan setiap tahap kepada kebutuhan fungsional yang dipenuhinya.



Gambar IV.1 Alur Proses Sistem ADADeH dari Unggahan hingga Penyajian Hasil

Tahap 1 sampai 7 berjalan otomatis tanpa keterlibatan pengguna, sesuai prinsip pipeline *end-to-end* pada Subsubbab IV.1.1. Apabila validasi pada tahap 1 tidak lolos,

Tabel IV.1 Pemetaan Tahap Alur Proses terhadap Kebutuhan Fungsional

Tahap	Deskripsi	KF Terkait
1	Validasi ukuran, durasi, dan dekode berkas	KF-02
2	Resampling ke *sampling rate* data latih, penyeragaman kanal	KF-03
3	Pemotongan rekaman menjadi K potongan tetap	KF-04
4	Ekstraksi mel-spectrogram skala dB per potongan	KF-05
5	Normalisasi menggunakan statistik data latih	KF-05, KNF-08
6	Inferensi skor deepfake p_k per potongan oleh CNN	KF-06
7	Agregasi skor $\bar{p}$ dan penentuan label dengan ambang τ	KF-07
8	Penyajian label, skor keyakinan, dan keterangan batasan	KF-08, KF-09, KF-10

misalnya berkas bukan audio atau berukuran melebihi batas KNF-03, sistem langsung menampilkan pesan kesalahan tanpa melanjutkan ke tahap berikutnya. Jika rekaman lolos validasi namun menunjukkan indikasi mutu rendah (durasi terlalu singkat, dominan hening, atau *clipping*), peringatan tersebut ditampilkan bersamaan dengan label pada tahap 8, bukan sebagai penolakan, karena hal ini memenuhi KF-10 sekaligus tetap memberikan indikasi awal kepada pengguna sesuai KP-07. Setelah hasil ditampilkan dan sesi pengguna berakhir, berkas unggahan serta data antara (waveform dan mel-spectrogram sementara) dihapus untuk memenuhi KF-12 dan KNF-05.

IV.1.3 Rancangan Keluaran bagi Pengguna

Mengingat pengguna sasaran adalah pengguna awam dan pendamping pelaporan (Subsubbab III.2.1), keluaran ADADeH dirancang dalam bahasa yang sederhana namun tetap menyertakan keterangan yang diperlukan untuk penggunaan hasilnya sebagai indikasi awal. Tabel IV.2 merinci setiap elemen keluaran.

Urutan penyajian mengikuti pedoman interaksi manusia–AI pada Subsubbab III.2.2 (KP-03): label dan skor ditampilkan paling menonjol, diikuti keterangan batasan dan peringatan mutu, dan ringkasan terunduh disediakan sebagai opsi tambahan bagi pendamping pelaporan. Dengan rancangan ini, desain sistem pada subbab ini telah mencakup seluruh kebutuhan fungsional KF-01 sampai KF-12 dan menjadi acuan langsung bagi arsitektur pada Subbab IV.2.

IV.2 Arsitektur Sistem

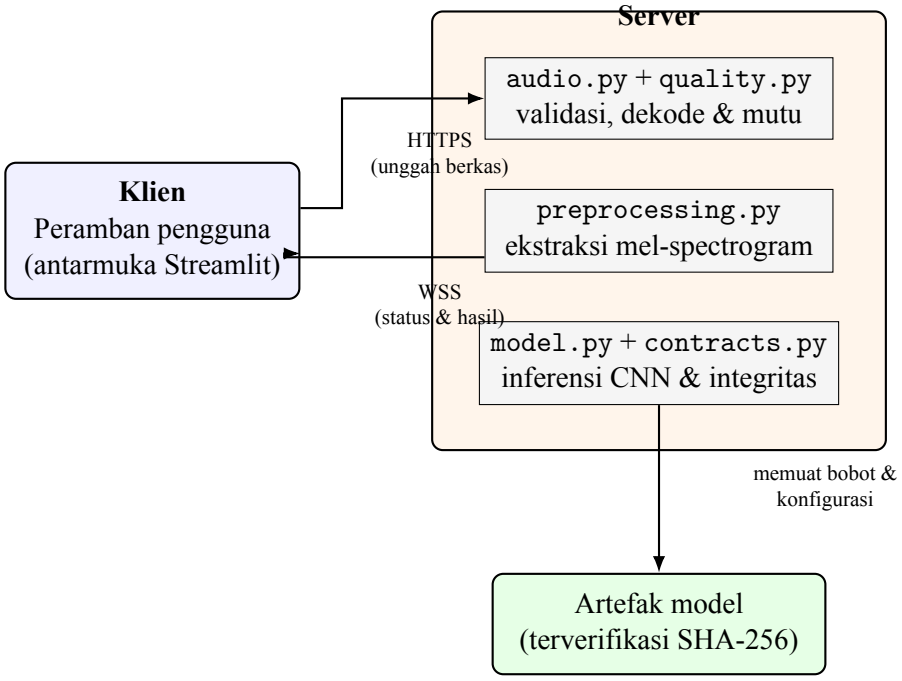
IV.2.1 Arsitektur Klien–Server

Mengikuti pilihan platform Streamlit pada Subbab II.3.2.5 dan II.3.3, ADADeH diimplementasikan dengan arsitektur klien–server dua lapis: peramban pengguna sebagai klien, dan aplikasi Python (`streamlit_app.py`, menjalankan paket `adadeh`)

Tabel IV.2 Elemen Keluaran ADADeH bagi Pengguna

Elemen	Isi	KF/KP Terkait
Label	"Terindikasi asli" atau "Terindikasi deepfake"	KF-08, KP-03
Skor keyakinan	Nilai $c = \max(\bar{p}, 1 - \bar{p})$ dalam persen	KF-08, KP-03
Keterangan batasan	Penegasan bahwa hasil bersifat indikasi awal, dilatih pada satu dataset berbahasa Inggris, dan bukan pengganti pemeriksaan forensik digital	KF-09, KP-04
Peringatan mutu	Catatan apabila rekaman terlalu singkat, dominan hening, atau terindikasi *clipping*	KF-10, KP-07
Ringkasan terunduh	Nama berkas, durasi, waktu pemeriksaan, nilai *hash* SHA-256, label, skor, versi model, dan keterangan batasan	KF-11, KP-09

sebagai server yang menjalankan seluruh pemrosesan audio dan inferensi model. Pada tahap purwarupa saat ini, klien dan server berjalan pada komputer yang sama (*localhost*); pembagian peran tetap mengikuti pola klien–server penuh agar aplikasi dapat dipindahkan ke penempatan jaringan tanpa mengubah struktur kode. Gambar IV.2 menggambarkan pembagian komponen tersebut beserta protokol komunikasi yang menghubungkannya, sejalan dengan Tabel II.4.



Gambar IV.2 Arsitektur Klien–Server ADADeH

Peran masing-masing komponen adalah sebagai berikut.

1. **Klien.** Peramban menampilkan halaman unggah, indikator kemajuan, dan hasil akhir. Klien tidak menyimpan maupun mengolah rekaman; seluruh berkas yang dipilih pengguna melalui `st.file_uploader` segera dikirim ke server melalui HTTP/HTTPS (Subbab II.3.4), sedangkan pembaruan tampilan selama pemrosesan berlangsung disalurkan melalui WebSocket/WSS (Subbab II.3.5).
2. **Modul validasi, dekode, dan pemeriksaan mutu (`audio.py`, `quality.py`).** Modul ini menjalankan Tahap 1 sampai 3 pada Gambar IV.1 sekaligus memperkuatnya dari sisi keamanan: dekode berkas dijalankan pada proses terpisah (*subprocess*) berbatas waktu, berkas M4A/ISO-BMFF diperiksa secara khusus agar tidak berisi rujukan data eksternal, dan rekaman yang lolos dekode diperiksa mutunya (hening, *clipping*, durasi) sebelum dilanjutkan ke tahap ekstraksi fitur. Rincian keduanya dibahas pada Subsubbab IV.3.9 dan IV.3.11.
3. **Modul ekstraksi mel-spectrogram (`preprocessing.py`).** Modul ini menjalankan Tahap 4 dan 5, membentuk mel-spectrogram skala dB dan menormalisasinya dengan statistik data latih yang tersimpan pada artefak konfigurasi.
4. **Modul inferensi dan integritas model (`model.py`, `contracts.py`).** Modul ini memuat bobot model terlatih, memverifikasi integritasnya (lihat butir Artefak model), dan menjalankan Tahap 6 dan 7, yaitu menghitung skor per potongan serta mengagregasinya menjadi skor dan label akhir melalui `prediction.py`.
5. **Artefak model.** Berkas `adadeh_cnn.keras` menyimpan arsitektur dan bobot CNN, sedangkan `meta.json` menyimpan seluruh parameter pra-pemrosesan, statistik normalisasi, dan ambang keputusan yang digunakan selama pelatihan (dibahas lebih lanjut pada Subsubbab IV.2.4). Keduanya dimuat bersama oleh server saat aplikasi dijalankan, sehingga parameter pelatihan dan inferensi selalu konsisten (KNF-08). Sebelum dimuat, nilai *hash* SHA-256 kedua berkas diverifikasi terhadap nilai yang tertanam pada kode aplikasi; apabila tidak cocok, aplikasi menghentikan pemuatan tanpa cadangan (*fallback*) apa pun, untuk mencegah model yang rusak atau telah diubah dipakai tanpa disadari (Subsubbab IV.3.10).
6. **Modul orkestrasi, antarmuka, dan pelaporan (`service.py`, `ui.py`, `reporting.py`).** Modul ini menghubungkan seluruh modul di atas menjadi satu alur `analyze_audio()`, menyajikannya pada antarmuka Streamlit (`st.file_uploader`, `st.status`), dan menyediakan ringkasan hasil yang dapat diunduh dalam format JSON maupun teks (Subsubbab IV.3.12).

IV.2.2 Arsitektur Paket Perangkat Lunak `adadeh`

Seluruh modul server pada Gambar IV.2 diwujudkan sebagai satu paket Python terinstal bernama `adadeh` (struktur `src`, didefinisikan melalui `pyproject.toml`), bukan sebagai satu berkas tunggal. Pemisahan ini memudahkan setiap tanggung jawab diuji secara independen (unit, integrasi, kesetaraan terhadap notebook, dan antarmuka – Subsubbab IV.3.14) serta memudahkan penelusuran kesalahan. Tabel IV.3 merinci sembilan modul penyusun paket tersebut, tanggung jawabnya, dan kebutuhan fungsional/nonfungsional yang dipenuhinya; urutannya sejalan dengan alur data pada Gambar IV.1, dari dekode berkas hingga penyajian serta pembersihan hasil.

IV.2.3 Arsitektur Model Klasifikasi CNN

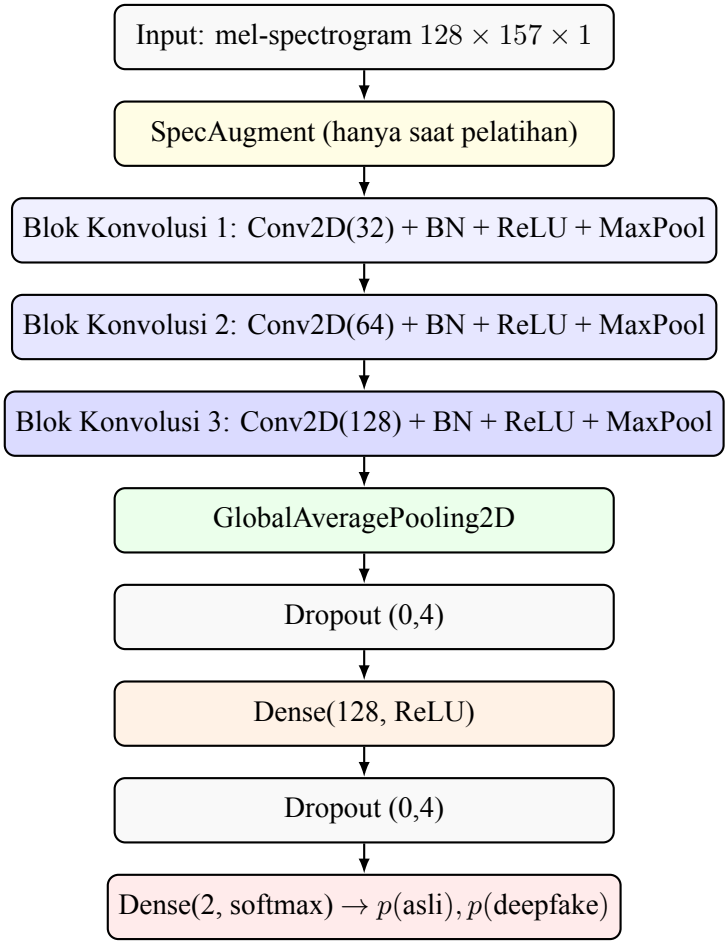
Mengikuti alternatif A2 pada Subbab III.3.4, model klasifikasi menerima mel-spectrogram satu kanal berukuran 128×157 sebagai citra dan memprosesnya melalui tiga blok konvolusi dengan jumlah filter yang meningkat (32, 64, 128), diikuti *global average pooling* dan dua lapisan *fully connected*. Gambar IV.3 menggambarkan susunan lapisan tersebut, dan Tabel IV.4 merinci setiap lapisan beserta dimensi keluaran dan jumlah parameternya.

Beberapa keputusan arsitektur berikut mendasari susunan pada Tabel IV.4.

- **SpecAugment sebagai augmentasi data.** Lapisan ini secara acak menutup (*masking*) sebagian pita frekuensi dan rentang waktu pada mel-spectrogram hanya selama pelatihan, sehingga model tidak terlalu bergantung pada satu pita frekuensi atau satu segmen waktu tertentu. Perlakuan ini sejalan dengan catatan invariansi pada Subsubbab II.1.2.2 bahwa pergeseran pada sumbu waktu tidak mengubah makna suara, sedangkan pergeseran pada sumbu frekuensi dapat mengubahnya; augmentasi karena itu diterapkan secara acak dan terbatas agar tidak merusak informasi frekuensi yang relevan.
- **Konvolusi tanpa *bias* sebelum normalisasi batch.** Setiap lapisan `Conv2D` diatur dengan `use_bias=False` karena *bias* akan dinetralkan oleh lapisan *batch normalization* yang mengikutinya, sehingga parameter tersebut dihilangkan untuk efisiensi.
- ***Global average pooling* alih-alih *flatten*.** Lapisan ini meringkas setiap peta fitur menjadi satu nilai rata-rata, menghasilkan vektor berukuran tetap (128) tanpa bergantung pada dimensi spasial hasil konvolusi. Pendekatan ini mengurangi jumlah parameter pada lapisan *fully connected* berikutnya dibandingkan *flatten*, sejalan dengan pertimbangan kemudahan implementasi (K4) dan ukuran model yang ringan pada Subbab III.3.2.
- **Keluaran dua kelas dengan *softmax*.** Model menghasilkan probabilitas un-

Tabel IV.3 Modul Penyusun Paket `adadeh`

Modul	**Tanggung Jawab**	**KF/KNF**
`audio.py`	Memeriksa amplop berkas, memvalidasi berkas M4A/ISO-BMFF agar tidak merujuk data eksternal, memanggil `ffprobe`, dan mendekode audio pada proses terpisah berbatas waktu (45 detik) dengan batas sumber daya	KF-02, KNF-03, KNF-06
`preprocessing.py`	Mengekstraksi fitur mel-spectrogram per potongan (`extract_features`), serupa dengan `gen_melspec_data` pada notebook (Kode Program IV.4)	KF-03, KF-04, KF-05
`quality.py`	Menilai mutu rekaman (`assess_quality`): hening, level rendah, *clipping*, dan durasi singkat, lalu menghasilkan peringatan yang menyertai hasil akhir	KF-10, KP-07
`contracts.py`	Mendefinisikan `ModelContract`: representasi tidak berubah (*frozen dataclass*) dari seluruh parameter pada `meta.json` agar parameter latih dan infer selalu identik	KNF-08
`model.py`	Memuat dan memverifikasi integritas artefak model (`ModelRuntime`), mendaftarkan ulang lapisan khusus SpecAugment, memvalidasi bentuk masukan secara ketat, dan menjalankan inferensi *batch* dengan kunci *thread*	KF-06, KNF-01, KNF-08
`prediction.py`	Mengagregasi skor per potongan menjadi skor berkas (Persamaan III.2) dan menentukan label akhir dengan operator ambang ketat ($\bar{p} > \tau$)	KF-07, KF-08
`service.py`	Mengorkestrasi seluruh modul di atas menjadi satu fungsi `analyze_audio()` yang dipanggil oleh antarmuka	KF-01 s.d. KF-08
`reporting.py`	Menghasilkan ringkasan hasil terunduh dalam format JSON (`to_json`) dan teks (`to_text`)	KF-11, KP-09
`ui.py`	Menyajikan antarmuka Streamlit: formulir unggah, indikator kemajuan, hasil, tombol unduh, serta memuat model satu kali melalui `st.cache_resource`	KF-01, KF-08, KF-09, KP-01



Gambar IV.3 Arsitektur Model CNN untuk Klasifikasi Mel-Spectrogram ADADeH

Tabel IV.4 Rincian Lapisan Model CNN ADADeH

Lapisan	**Konfigurasi**	**Dimensi Keluaran**	**Parameter**
Input	–	$128 \times 157 \times 1$	0
SpecAugment	`time_mask`=20, `freq_mask`=16	$128 \times 157 \times 1$	0
Conv2D + BN + ReLU	32 filter, kernel 3×3, *same*	$128 \times 157 \times 32$	288 + 64
MaxPool2D	ukuran 2×2	$64 \times 78 \times 32$	0
Conv2D + BN + ReLU	64 filter, kernel 3×3, *same*	$64 \times 78 \times 64$	18.432 + 128
MaxPool2D	ukuran 2×2	$32 \times 39 \times 64$	0
Conv2D + BN + ReLU	128 filter, kernel 3×3, *same*	$32 \times 39 \times 128$	73.728 + 256
MaxPool2D	ukuran 2×2	$16 \times 19 \times 128$	0
GlobalAveragePooling2D	–	128	0
Dropout	p=0,4	128	0
Dense	128 unit, ReLU	128	16.512
Dropout	p=0,4	128	0
Dense	2 unit, softmax	2	258
Total parameter terlatih (dihitung dari definisi lapisan)			≈109.666
Total parameter tidak terlatih (statistik *batch normalization*)			448

tuk kelas *real* dan *fake* sekaligus ($p(\text{asli}) + p(\text{deepfake}) = 1$), konsisten dengan definisi kelas pada Subsubbab II.4.1. Probabilitas kelas *fake* inilah yang menjadi p pada Persamaan II.3 dan II.4.

Dengan total parameter terlatih sekitar 110 ribu, model ini jauh lebih ringan dibandingkan model *self-supervised* pralatih pada alternatif A4 (sekitar 300 juta parameter, Subsubbab III.3.1), sehingga selaras dengan keterbatasan perangkat pelatihan dan inferensi pada Subbab II.3.1.

IV.2.4 Arsitektur Konfigurasi dan Artefak Model

Untuk memenuhi KNF-08, seluruh parameter yang memengaruhi bentuk masukan model disimpan dalam satu berkas konfigurasi (`meta.json`) yang menyertai berkas bobot model (`adadeh_cnn.keras`). Tabel IV.5 merinci isi berkas tersebut, sesuai dengan artefak yang benar-benar disertakan pada aplikasi. Dengan skema ini, modul inferensi pada Gambar IV.2 tidak pernah menuliskan ulang nilai parameter secara manual, melainkan selalu membacanya dari `meta.json` melalui `ModelContract` (`contracts.py`) sehingga rekaman baru diproses dengan parameter yang identik dengan data latih.

Tabel IV.5 Isi Berkas Konfigurasi `meta.json`

Kunci	Nilai	Keterangan
`mu`	$-28{,}459316$	Rata-rata mel-spectrogram data latih (statistik normalisasi)
`sd`	$16{,}406836$	Deviasi standar mel-spectrogram data latih
`threshold`	$0{,}51$	Ambang keputusan τ hasil pencarian pada data validasi
`global_sr`	16000	*Sampling rate* acuan (Hz)
`chunk_dur`	40000	Panjang satu potongan (sampel)
`step`	20000	Jarak antarpotongan (sampel)
`n_fft`	1024	Ukuran jendela STFT
`hop_length`	256	Jarak antar-bingkai STFT
`n_mels`	128	Jumlah *mel bin*
`classes`	`["real", "fake"]`	Pemetaan indeks kelas keluaran model

Selain memuat `meta.json`, modul `model.py` menghitung nilai *hash* SHA-256 dari berkas `adadeh_cnn.keras` dan `meta.json` saat aplikasi dimulai, lalu membandingkannya dengan nilai *hash* yang tertanam secara tetap (*hardcoded*) pada kode aplikasi (`MODEL_SHA256`, `META_SHA256`). Apabila salah satu nilai tidak cocok, aplikasi menolak memuat model dan tidak menyediakan cadangan otomatis ke model lain,

sehingga pengguna tidak akan pernah menerima hasil dari artefak yang tidak diketahui asalnya atau telah berubah – pemenuhan KNF-08 yang lebih kuat dibandingkan sekadar membaca ulang parameter dari `meta.json`.

Dengan arsitektur ini, Subbab IV.2 telah menjabarkan baik struktur komponen perangkat lunak (klien, server, paket `adadeh`) maupun struktur model klasifikasi yang dijalankan di dalamnya. Subbab berikutnya menjelaskan bagaimana komponen-komponen ini diimplementasikan.

IV.3 Implementasi

Implementasi purwarupa ADADeH mencakup seluruh tahap pada pendekatan CRISP-DM yang diacu Bab I – penyiapan data, pra-pemrosesan, pemodelan, dan evaluasi – sekaligus pembangunan aplikasi yang menjalankan model tersebut bagi pengguna akhir. Subbab ini menjelaskan kedua bagian tersebut secara berurutan. Subsubbab IV.3.1 sampai IV.3.8 menjelaskan pengembangan model yang diwujudkan pada notebook pengembangan yang dijalankan di Kaggle Notebook, sesuai perangkat pelatihan model yang dijelaskan pada Subsubbab II.3.1.1. Subsubbab IV.3.9 sampai IV.3.14 menjelaskan pembangunan aplikasi web yang mengemas model tersebut sebagai paket Python `adadeh` dan antarmuka Streamlit yang dapat diakses pengguna, sesuai arsitektur pada Subbab IV.2.

IV.3.1 Lingkungan dan Pustaka Implementasi

Implementasi dikerjakan pada Kaggle Notebook dengan akselerator GPU NVIDIA Tesla T4, sejalan dengan Tabel II.1. Pustaka yang digunakan pada tahap akuisisi data, pra-pemrosesan, pemodelan, dan evaluasi dirangkum pada Tabel IV.6.

IV.3.1.0.1 Catatan penyesuaian implementasi. Subsubbab II.3.2.3 merencanakan PyTorch sebagai pustaka utama pembangunan model CNN. Pada pelaksanaannya, PyTorch (melalui `torchaudio`) tetap dipakai untuk mendekode dan menyeragamkan *sampling rate* audio, tetapi model CNN akhirnya dibangun dan dilatih dengan TensorFlow/Keras. Penyesuaian ini diambil karena API `Sequential` Keras mempermudah penyusunan blok konvolusi yang berulang (Subsubbab IV.2.3) serta menyediakan *callback* `EarlyStopping` dan `ReduceLROnPlateau` secara siap pakai. Penyesuaian pustaka ini tidak mengubah rancangan arsitektur maupun alur data yang telah ditetapkan pada Subbab II.2 dan Subbab IV.1, dan sejalan dengan sifat iteratif pendekatan CRISP-DM pada Bab I yang memungkinkan penyempurnaan komponen sebelum integrasi akhir.

Tabel IV.6 Pustaka yang Digunakan pada Implementasi ADADeH

Pustaka	**Peran pada Implementasi**
`datasets` (Hugging Face)	Mengunduh dan memuat dataset deepfake audio langsung dari Hugging Face Hub
`torchaudio`, `torch`	Mendekode gelombang audio dan melakukan *resampling* ke *sampling rate* global
`librosa`	Menghitung STFT, filter bank mel, dan konversi ke skala dB untuk membentuk mel-spectrogram (Subbab II.3.2.2)
`NumPy`	Menyimpan dan menggabungkan larik mel-spectrogram serta label (Subbab II.3.2.2)
`TensorFlow/Keras`	Membangun, melatih, dan menyimpan model CNN
`scikit-learn`	Menghitung *class weight*, metrik evaluasi, dan mencari ambang keputusan terbaik (Subbab II.3.2.4)
`Matplotlib`	Memvisualisasikan *waveform* dan mel-spectrogram selama eksplorasi data

IV.3.2 Implementasi Akuisisi Data

Dataset diperoleh langsung dari Hugging Face Hub melalui pustaka `datasets` menggunakan satu baris kode:

Listing IV.1 Akuisisi dataset dari Hugging Face Hub

```
from datasets import load_dataset

ds = load_dataset("garystafford/deepfake-audio-detection", split="
    train")
```

Dataset ini berisi 1.866 rekaman dengan dua fitur: `audio` (gelombang suara) dan `label` (`ClassLabel` dengan nilai `real` dan `fake`), sesuai dengan definisi kelas biner pada Subsubbab II.4.1. Pemeriksaan awal terhadap rekaman pertama dan terakhir dalam dataset menunjukkan bahwa format, jumlah kanal, dan *sampling rate* rekaman tidak seragam antarberkas (misalnya satu rekaman berkanal ganda pada 44.100 Hz, sedangkan rekaman lain berkanal tunggal pada 16.000 Hz), sehingga penyeragaman pada tahap pra-pemrosesan menjadi suatu keharusan, bukan sekadar pilihan desain.

Statistik durasi dan *sampling rate* atas seluruh rekaman dihitung untuk menentukan parameter pemotongan pada Subsubbab IV.3.4. Hasilnya disajikan pada Tabel IV.7.

Jumlah rekaman per kelas secara menyeluruh tidak dihitung secara eksplisit pada notebook. Namun, komposisi data uji yang persis seimbang (140 rekaman asli dan

Tabel IV.7 Statistik Durasi dan *Sampling Rate* Dataset ADADeH

Aspek	**Statistik**	**Nilai**
Durasi rekaman (detik)	Rata-rata	4,195
Durasi rekaman (detik)	Minimum	2,500
Durasi rekaman (detik)	Maksimum	12,526
Sampling rate (Hz)	Minimum	16.000
Sampling rate (Hz)	Maksimum	44.100

140 rekaman deepfake dari 280 berkas, lihat Tabel IV.8) menunjukkan bahwa dataset sumber kemungkinan besar disusun dengan proporsi label yang seimbang pada tingkat berkas, meskipun proporsi tersebut tidak otomatis terbawa ke tingkat potongan setelah pemotongan (Subsubbab IV.3.4).

IV.3.3 Implementasi Pembagian Data

Sejalan dengan prinsip pemisahan data pada Subsubbab II.4.3, pembagian data latih, validasi, dan uji dilakukan pada tingkat *berkas* (bukan tingkat potongan), dengan rasio mendekati 70:15:15 dari 1.866 rekaman, menghasilkan 1.306 berkas latih, 280 berkas validasi, dan 280 berkas uji. Pembagian pada tingkat berkas ini memastikan seluruh potongan yang berasal dari satu rekaman berada pada subset data yang sama, sehingga mencegah kebocoran data (*data leakage*) sebagaimana ditekankan pada Subsubbab II.4.3.

Mel-spectrogram untuk setiap subset dibentuk melalui fungsi `build_split` yang memanggil fungsi ekstraksi fitur (Subsubbab IV.3.4) untuk setiap berkas, lalu menggabungkan hasilnya menjadi satu larik:

Listing IV.2 Pembangunan subset data latih, validasi, dan uji

```
def build_split(ds, indices):
    parts = [gen_melspec_data(ds, int(i)) for i in tqdm(indices)]
    out = {k: np.concatenate([p[k] for p in parts]) for k in ("
    array", "label")}
    out["file_id"] = np.concatenate(
        [np.full(len(p["label"]), int(i)) for p, i in zip(parts,
    indices)]
    )
    return out

train = build_split(ds, tr)
val   = build_split(ds, va)
test  = build_split(ds, te)
```

Larik `file_id` pada setiap subset menyimpan indeks berkas asal setiap potongan dan menjadi kunci bagi agregasi skor per berkas pada Subsubbab IV.3.7 (KF-07). Hasil pembagian data pada tingkat potongan disajikan pada Tabel IV.8; data ini kemudian disimpan ke berkas `.npz` terkompresi (`train.npz`, `val.npz`, `test.npz`) agar dapat dimuat ulang tanpa mengulang ekstraksi fitur.

Tabel IV.8 Komposisi Data Latih, Validasi, dan Uji (Tingkat Berkas dan Potongan)

Subset	**Berkas**	**Potongan**	**Potongan asli (*real*)**	**Potongan deepfake (*fake*)**
Latih	1.306	2.415	1.442	973
Validasi	280	512	301	211
Uji	280	507	305	202

Tabel IV.8 memperlihatkan bahwa meskipun pembagian berkas seimbang, jumlah potongan antarkelas pada data latih tidak seimbang (1.442 berbanding 973), karena rekaman asli dan deepfake tidak selalu menghasilkan jumlah potongan yang sama (dibatasi maksimum 4 potongan per rekaman pada Subsubbab IV.3.4). Ketidakseimbangan inilah yang ditangani melalui *class weight* pada tahap pelatihan (Subsubbab IV.3.6).

IV.3.4 Implementasi Pra-Pemrosesan dan Pemotongan Audio

Parameter pemotongan ditentukan dari statistik dataset pada Tabel IV.7: *sampling rate* global sr_g diambil dari nilai minimum (16.000 Hz) agar seluruh rekaman dapat diturunkan (*downsample*) tanpa perlu menaikkan *sampling rate* rekaman bermutu rendah, dan durasi satu potongan diambil dari durasi rekaman minimum (2,5 detik) agar setiap rekaman dalam dataset dapat menghasilkan sedikitnya satu potongan penuh. Tabel IV.9 merangkum parameter yang dihasilkan.

Listing IV.3 Penentuan parameter pemotongan audio

```
global_sr   = minimum_spg_rate            # 16000
chunk_dur   = int(minimum_duration * global_sr)  # 40000 sampel
    (2,5 detik)
overlap     = int(1.25 * global_sr)              # 20000 sampel
    (1,25 detik)
step        = chunk_dur - overlap                # 20000 sampel
    (1,25 detik)
max_chunk   = 4
```

Setiap rekaman diturunkan menjadi satu kanal (*mono*) dengan merata-ratakan kanal, kemudian di-*resample* ke sr_g apabila *sampling rate* aslinya berbeda, sebelum dipotong menjadi maksimum empat potongan yang saling tumpang-tindih sebesar

Tabel IV.9 Parameter Pemotongan dan Ekstraksi Mel-Spectrogram

Parameter	**Keterangan**	**Nilai**
sr_g	*Sampling rate* global	16.000 Hz
`chunk_dur`	Durasi satu potongan	40.000 sampel (2,5 s)
`step`	Jarak antarpotongan (tumpang-tindih 50%)	20.000 sampel (1,25 s)
`max_chunk`	Jumlah maksimum potongan per rekaman	4
`n_fft`	Ukuran jendela STFT	1.024
`hop_length`	Jarak antar-bingkai STFT	256
`n_mels`	Jumlah *mel bin*	128
f_{max}	Batas atas frekuensi mel	$sr_g/2 = 8.000$ Hz

50%:

Listing IV.4 Fungsi ekstraksi mel-spectrogram per potongan

```
def gen_melspec_data(ds, index, step=step, chunk_dur=chunk_dur,
                     max_chunk=max_chunk, global_sr=global_sr,
                     n_fft=1024, hop_length=256, n_mels=128):
    s = ds[index]["audio"].get_all_samples()
    wav = s.data.mean(0, keepdim=True)
    if s.sample_rate != global_sr:
        wav = torchaudio.functional.resample(wav, s.sample_rate,
   global_sr)
    y = wav.squeeze(0).numpy()
    label = ds[index]["label"]

    n = (len(y) - chunk_dur) // step + 1
    n = min(n, max_chunk)

    arrays = []
    for i in range(n):
        start, end = i * step, i * step + chunk_dur
        chunk = y[start:end]
        mel = librosa.feature.melspectrogram(
            y=chunk, sr=global_sr, n_fft=n_fft, hop_length=
   hop_length,
            n_mels=n_mels, fmax=global_sr // 2)
        mel_db = librosa.power_to_db(mel, ref=1.0, top_db=80)
        arrays.append(mel_db)

    return {
        "array": np.stack(arrays).astype(np.float32),   # (n, 128,
    157)
        "label": np.full(n, label, dtype=np.int64),     # (n,)
    }
```

Setiap potongan menghasilkan satu mel-spectrogram berukuran 128×157 (128 *mel bin* terhadap 157 bingkai waktu), sesuai dengan rumusan matriks $X \in \mathbb{R}^{M \times T}$ pada Subbab II.2. Fungsi ini sekaligus mengimplementasikan KF-03, KF-04, dan KF-05 pada tingkat satu rekaman, dan dipanggil berulang oleh `build_split` (Kode Program IV.2) untuk membentuk subset latih, validasi, dan uji.

IV.3.5 Implementasi Model CNN

Model CNN dibangun sesuai rancangan pada Subsubbab IV.2.3 menggunakan API `Sequential` Keras. Lapisan SpecAugment diimplementasikan sebagai lapisan khusus yang menerapkan *masking* acak pada sumbu frekuensi dan waktu hanya ketika `training=True`:

Listing IV.5 Lapisan SpecAugment

```
class SpecAugment(layers.Layer):
    def __init__(self, time_mask=20, freq_mask=16, **kw):
        super().__init__(**kw)
        self.tm, self.fm = time_mask, freq_mask

    def _mask(self, B, n, limit, shape):
        w = tf.random.uniform([B, 1, 1, 1], 0, limit, dtype=tf.
    int32)
        s = tf.cast(tf.random.uniform([B, 1, 1, 1]) * tf.cast(n -
    w, tf.float32), tf.int32)
        r = tf.reshape(tf.range(n), shape)
        return (r >= s) & (r < s + w)

    def call(self, x, training=None):
        if not training:
            return x
        B, F, T = tf.shape(x)[0], tf.shape(x)[1], tf.shape(x)[2]
        m = self._mask(B, F, self.fm, [1, -1, 1, 1]) | self._mask(
    B, T, self.tm, [1, 1, -1, 1])
        return tf.where(m, tf.zeros_like(x), x)
```

Blok konvolusi yang berulang disusun melalui fungsi pembantu `block`, lalu dirangkai menjadi model lengkap sebagaimana Kode Program IV.6, sesuai dengan Gambar IV.3 dan Tabel IV.4.

Listing IV.6 Definisi model CNN ADADeH

```
def block(f):
    return [layers.Conv2D(f, 3, padding="same", use_bias=False),
            layers.BatchNormalization(momentum=0.9),
            layers.ReLU(),
```

```
            layers.MaxPool2D(2)]

model = models.Sequential([
    layers.Input(shape=X_train.shape[1:]),        # (128, 157, 1)
    SpecAugment(),
    *block(32), *block(64), *block(128),
    layers.GlobalAveragePooling2D(),
    layers.Dropout(0.4),
    layers.Dense(128, activation="relu"),
    layers.Dropout(0.4),
    layers.Dense(len(classes), activation="softmax"),
])

model.compile(optimizer=tf.keras.optimizers.Adam(3e-4),
              loss="sparse_categorical_crossentropy",
              metrics=["accuracy"])
```

Sebelum dilatihkan ke model, mel-spectrogram dinormalisasi menggunakan rata-rata (μ) dan deviasi standar (σ) yang dihitung *hanya* dari data latih, lalu diterapkan pula pada data validasi dan uji, sesuai prinsip pada Subsubbab II.4.3 agar statistik data uji tidak memengaruhi pembentukan model:

Listing IV.7 Normalisasi mel-spectrogram

```
mu, sd = X_train.mean(), X_train.std()
X_train, X_val, X_test = [(x - mu) / (sd + 1e-8) for x in (X_train
    , X_val, X_test)]
```

IV.3.6 Implementasi Pelatihan Model

Ketidakseimbangan kelas pada data latih (Tabel IV.8) ditangani dengan *class weight* seimbang (*balanced*) dari scikit-learn, yang menghasilkan bobot sekitar $0{,}8374$ untuk kelas *real* dan $1{,}2410$ untuk kelas *fake* – kelas minoritas memperoleh bobot kerugian yang lebih besar agar kesalahan pada kelas tersebut lebih memengaruhi pembaruan parameter model. Pelatihan dijalankan dengan konfigurasi pada Tabel IV.10.

Listing IV.8 Pelatihan model CNN dengan *class weight* dan *callback*

```
cw = compute_class_weight("balanced", classes=np.unique(y_train),
    y=y_train)
class_weight = dict(enumerate(cw))

cbs = [
    callbacks.EarlyStopping(monitor="val_loss", patience=10,
```

```
                                restore_best_weights=True),
    callbacks.ReduceLROnPlateau(monitor="val_loss", factor=0.5,
                                    patience=3, min_lr=1e-5),
]

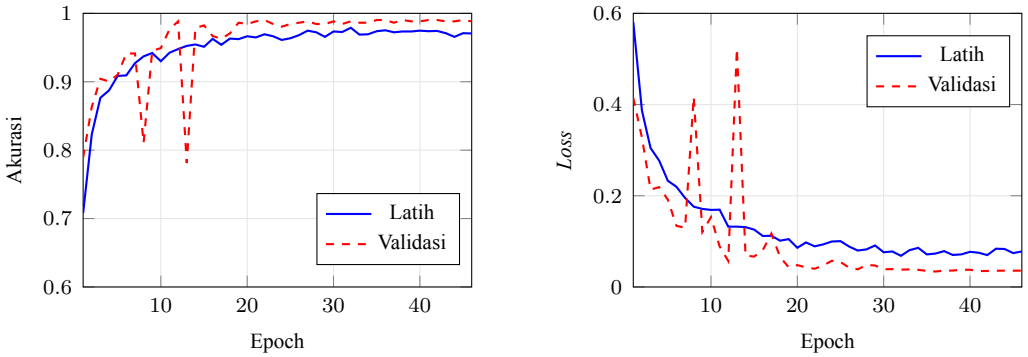
history = model.fit(X_train, y_train, epochs=60, batch_size=64,
                    validation_data=(X_val, y_val),
                    class_weight=class_weight, callbacks=cbs)
```

Tabel IV.10 Konfigurasi Pelatihan Model CNN ADADeH

Pengaturan	**Nilai**
Optimizer	Adam, *learning rate* awal 3×10^{-4}
Fungsi *loss*	*Sparse categorical cross-entropy*
Ukuran *batch*	64
Epoch maksimum	60
Class weight	*Real*: $0{,}8374$; *Fake*: $1{,}2410$
`EarlyStopping`	Memantau `val_loss`, kesabaran 10 epoch, mengembalikan bobot terbaik
`ReduceLROnPlateau`	Memantau `val_loss`, faktor $0{,}5$, kesabaran 3 epoch, LR minimum 10^{-5}

Dengan konfigurasi tersebut, pelatihan berhenti secara otomatis pada epoch ke-46 (dari 60 epoch maksimum) karena *validation loss* tidak membaik lagi selama 10 epoch berturut-turut setelah pencapaian terbaiknya. *Learning rate* diturunkan sebanyak lima kali oleh `ReduceLROnPlateau` (dari 3×10^{-4} menjadi 10^{-5}) seiring *validation loss* mendatar. Bobot terbaik yang dikembalikan berasal dari epoch ke-36, dengan *validation accuracy* $99{,}02\%$ dan *validation loss* $0{,}0339$. Gambar IV.4 menunjukkan kurva akurasi dan *loss* selama 46 epoch pelatihan tersebut, diambil langsung dari log pelatihan pada notebook.

Gambar IV.4 memperlihatkan dua hal yang relevan dengan keputusan desain. Pertama, *validation loss* sempat melonjak tajam pada beberapa epoch (misalnya epoch ke-8 dan ke-13) sebelum kembali turun, yang merupakan pola umum pada pelatihan dengan *learning rate* yang belum diturunkan serta augmentasi SpecAugment yang membuat setiap *batch* pelatihan sedikit berbeda. Kedua, sejak *learning rate* mulai diturunkan (epoch ke-16 dan seterusnya), kurva validasi menjadi jauh lebih stabil, menunjukkan bahwa `ReduceLROnPlateau` berperan penting dalam mencapai konvergensi yang stabil pada model berukuran ringan ini.



(a) Kurva akurasi (b) Kurva *loss*

Gambar IV.4 Kurva Akurasi dan *Loss* Selama Pelatihan Model CNN ADADeH (46 Epoch)

IV.3.7 Implementasi Evaluasi Model

Evaluasi dilakukan dalam dua tahap sesuai KF-07: pencarian ambang keputusan τ pada data validasi, kemudian pengukuran kinerja pada data uji baik pada tingkat potongan maupun tingkat berkas (hasil agregasi, Persamaan III.2).

Listing IV.9 Pencarian ambang keputusan dan evaluasi model

```
pv = model.predict(X_val)[:, 1]
ths = np.linspace(0.05, 0.95, 91)
best_t = ths[np.argmax([f1_score(y_val, pv > t, average="macro")
    for t in ths])]

pt = model.predict(X_test)[:, 1]
pred = (pt > best_t).astype(int)

fid = test["file_id"]
files = np.unique(fid)
p_file = np.array([pt[fid == f].mean() for f in files])      #
    Persamaan III.2
y_file = np.array([y_test[fid == f][0] for f in files])
pred_file = (p_file > best_t).astype(int)
```

Pencarian pada 91 nilai ambang antara 0,05 dan 0,95 menghasilkan $\tau = 0{,}51$ sebagai ambang yang memaksimalkan F1 makro pada data validasi. Nilai inilah yang disimpan pada `meta.json` (Tabel IV.5) dan digunakan pada Persamaan II.4 saat purwarupa menentukan label akhir. Perhitungan `p_file` pada baris ke-8 kode di atas secara langsung mengimplementasikan Persamaan III.2, yaitu merata-ratakan skor seluruh potongan suatu berkas (diidentifikasi melalui `file_id`) menjadi satu skor berkas $\bar{p}$.

Tabel IV.11 dan IV.12 menyajikan hasil evaluasi pada data uji untuk masing-masing tingkat.

Tabel IV.11 Hasil Evaluasi pada Data Uji – Tingkat Potongan ($n = 507$)

Kelas	**Presisi**	***Recall***	**F1**	**Dukungan**
Real (asli)	1,00	0,98	0,99	305
Fake (deepfake)	0,97	1,00	0,99	202
Akurasi keseluruhan	0,99			
ROC-AUC	0,9987			
EER	0,99%			

Matriks konfusi (baris: label asli; kolom: label prediksi; urutan *real*, *fake*): $\begin{pmatrix} 299 & 6 \\ 0 & 202 \end{pmatrix}$

Tabel IV.12 Hasil Evaluasi pada Data Uji – Tingkat Berkas ($n = 280$, Agregasi Persamaan III.2)

Kelas	**Presisi**	***Recall***	**F1**	**Dukungan**
Real (asli)	1,00	0,97	0,99	140
Fake (deepfake)	0,97	1,00	0,99	140
Akurasi keseluruhan	0,99			
ROC-AUC	0,9971			
EER	1,43%			

Matriks konfusi (baris: label asli; kolom: label prediksi; urutan *real*, *fake*): $\begin{pmatrix} 136 & 4 \\ 0 & 140 \end{pmatrix}$

Dibandingkan dengan kriteria keberhasilan pada Bab I (akurasi dan F1 minimal 90%), hasil pada Tabel IV.11 dan IV.12 menunjukkan pencapaian yang jauh melampaui target tersebut, baik pada tingkat potongan maupun tingkat berkas. Nilai ROC-AUC yang sangat tinggi (0,9987 pada tingkat potongan dan 0,9971 pada tingkat berkas) juga mengindikasikan bahwa kedua kelas dapat dipisahkan dengan sangat baik pada hampir seluruh nilai ambang. Baris EER pada kedua tabel mengonfirmasi hal tersebut secara langsung: pada notebook pengembangan, EER belum dihitung secara eksplisit, sehingga nilai pada Tabel IV.11 dan IV.12 diperoleh kemudian melalui modul `evaluation.py` pada aplikasi (dibahas pada Subsubbab IV.3.13), yang menghitung EER melalui interpolasi linear pada kurva ROC sesuai Persamaan II.7, yaitu titik potong ketika *false positive rate* sama dengan *false negative rate*. EER sebesar 0,99% pada tingkat potongan dan 1,43% pada tingkat berkas menunjukkan bahwa kesalahan klasifikasi hanya terjadi pada sebagian kecil dari seratus rekaman, konsisten dengan cara verifikasi KNF-01 pada Tabel III.8.

Matriks konfusi pada kedua tabel juga menunjukkan pola yang konsisten: tidak ada rekaman deepfake yang salah diklasifikasikan sebagai asli (*false negative* nol) pada kedua tingkat evaluasi, sedangkan sejumlah kecil rekaman asli diklasifikasikan sebagai deepfake (6 potongan dan 4 berkas). Dari sisi tujuan ADADeH untuk memberi indikasi awal bagi korban (Bab I), pola kesalahan seperti ini relatif lebih aman dibandingkan sebaliknya, karena model pada tahap pengujian ini cenderung lebih waspada terhadap deepfake daripada meloloskannya sebagai suara asli.

IV.3.8 Implementasi Penyimpanan Artefak Model

Setelah pelatihan dan evaluasi selesai, model, riwayat pelatihan, dan konfigurasi disimpan sebagai tiga berkas terpisah agar dapat dimuat kembali oleh purwarupa (Subbab IV.2.4):

Listing IV.10 Penyimpanan model dan konfigurasi

```
model.save(out + "adadeh_cnn.keras")

with open(out + "history.json", "w") as f:
    json.dump({k: [float(v) for v in vals] for k, vals in history.
    history.items()}, f)

meta = {
    "mu": float(mu), "sd": float(sd), "threshold": float(best_t),
    "global_sr": int(global_sr), "chunk_dur": int(chunk_dur), "
    step": int(step),
    "n_fft": 1024, "hop_length": 256, "n_mels": 128, "classes":
    classes,
}
with open(out + "meta.json", "w") as f:
    json.dump(meta, f)
```

Berkas `adadeh_cnn.keras` (sekitar 1,4 MB) menyimpan arsitektur dan bobot model secara utuh, `history.json` menyimpan riwayat metrik setiap epoch untuk keperluan dokumentasi dan analisis lanjutan (termasuk Gambar IV.4), dan `meta.json` menyimpan seluruh parameter yang telah dibahas pada Tabel IV.5. Ketiga berkas inilah yang kemudian disertakan sebagai artefak aplikasi dan dimuat oleh modul inferensi pada arsitektur klien–server (Subbab IV.2.1). Subsubbab berikut menjelaskan bagaimana artefak ini diintegrasikan ke dalam aplikasi Streamlit yang telah dibangun, mulai dari validasi audio hingga pengujian perangkat lunak.

IV.3.9 Implementasi Validasi dan Keamanan Audio

Berbeda dengan notebook pengembangan yang membaca berkas dari dataset yang telah terpercaya, aplikasi harus menerima unggahan dari pengguna mana pun, sehingga modul `audio.py` dirancang dengan asumsi bahwa setiap berkas masukan dapat berisi data yang keliru atau berbahaya. Validasi dijalankan berlapis sebelum satu *sample* audio pun diproses lebih jauh:

1. **Pemeriksaan amplop berkas.** Ukuran dan beberapa *byte* awal berkas diperiksa untuk memastikan jenis berkas sesuai dengan yang diklaim, sebelum berkas dibuka oleh pustaka dekode mana pun.
2. **Pengerasan khusus untuk M4A/ISO-BMFF.** Berkas dengan wadah ISO-BMFF (termasuk M4A) diurai secara manual pada tingkat *atom* untuk menolak berkas yang merujuk data pada lokasi eksternal (*external data reference*), suatu teknik yang dapat dipakai untuk membaca berkas lain di luar yang diunggah pengguna.
3. **Pemeriksaan melalui `ffprobe`.** Metadata audio (durasi, jumlah kanal, *sampling rate*) diperiksa melalui `ffprobe` sebelum dekode penuh dijalankan, sehingga berkas yang jelas tidak valid dapat ditolak lebih awal (KF-02).
4. **Dekode terisolasi proses.** Dekode audio sesungguhnya (melalui TorchCodec) dijalankan pada *subprocess* terpisah dengan batas waktu 45 detik dan batas sumber daya (memori dan CPU), sehingga berkas yang dirancang untuk menyebabkan dekoder mengonsumsi sumber daya tak terbatas (*denial-of-service*) tidak dapat memengaruhi proses utama aplikasi (KNF-03, KNF-06).

Keempat lapis ini menjalankan Tahap 1 pada Gambar IV.1 secara lebih ketat dibandingkan asumsi awal pada Bab III, yang hanya menyebutkan validasi ukuran dan durasi berkas. Penguatan ini diambil karena purwarupa pada Bab III dan IV versi notebook belum memperhitungkan bahwa aplikasi akan diakses secara langsung oleh pengguna yang tidak dikenal, sehingga permukaan serangan (*attack surface*) perlu dipersempit sebelum aplikasi dipertimbangkan untuk diakses publik (lihat pembatasan pada Subbab IV.4).

IV.3.10 Implementasi Runtime Model dan Verifikasi Integritas

Modul `contracts.py` mendefinisikan `ModelContract` sebagai *frozen dataclass* yang memuat seluruh kunci pada Tabel IV.5, beserta metode `normalize()` yang mengonversi nilai JSON mentah (misalnya *string* atau *integer*) menjadi tipe data Python yang konsisten dengan yang dipakai pada notebook (*float* untuk `mu`/`sd`/`threshold`, *int* untuk parameter pemotongan dan STFT). Kontrak ini kemudian dipakai oleh `model.py` pada kelas `ModelRuntime`, yang bertanggung jawab atas tiga hal:

- **Verifikasi integritas artefak.** Sebagaimana dijelaskan pada Subsubbab IV.2.4, *hash* SHA-256 berkas model dan konfigurasi dihitung saat pemuatan dan dibandingkan dengan konstanta `MODEL_SHA256` dan `META_SHA256` yang tertanam pada kode; ketidaksesuaian menghentikan pemuatan tanpa cadangan otomatis.
- **Pendaftaran ulang lapisan khusus.** Karena model memakai lapisan `SpecAugment` buatan sendiri (Kode Program IV.5), modul ini mendaftarkan kembali lapisan tersebut sebagai *custom object* Keras agar `adadeh_cnn.keras` dapat dimuat ulang tanpa kesalahan *deserialization*.
- **Validasi bentuk masukan secara ketat.** Sebelum memanggil `model.predict`, bentuk larik masukan diperiksa agar persis $(\cdot, 128, 157, 1)$ sesuai Tabel IV.4; ketidaksesuaian bentuk menimbulkan kesalahan eksplisit alih-alih dibiarkan menghasilkan keluaran yang salah secara diam-diam, sejalan dengan prinsip konsistensi parameter pada Subsubbab IV.1.1.

Inferensi dijalankan dalam mode *batch* dengan penguncian (*thread lock*), sehingga pemanggilan model dari beberapa permintaan pengguna secara bersamaan tidak saling bertabrakan pada objek model Keras yang sama (KNF-06). Label akhir dan skor keyakinan kemudian ditentukan oleh `prediction.py` melalui fungsi `classify_scores()`, yang menerapkan operator ambang ketat lebih besar ($\bar{p} > \tau$), persis seperti pencarian ambang pada Kode Program IV.9, dan menghasilkan label teks "Terindikasi deepfake" atau "Terindikasi asli" sesuai rancangan pada Tabel IV.2.

IV.3.11 Implementasi Pemeriksaan Mutu Rekaman

Modul `quality.py` mengimplementasikan `QualityPolicy`, yaitu kebijakan ambang tetap yang menjadi dasar peringatan mutu pada Tabel IV.2 (KF-10, KP-07). Fungsi `assess_quality()` memeriksa empat kondisi atas rekaman yang telah lolos validasi pada Subsubbab IV.3.9, dirangkum pada Tabel IV.13.

Tabel IV.13 Kebijakan Pemeriksaan Mutu Rekaman (`QualityPolicy`)

Kondisi	**Ambang**
Hening (*silence*)	Sedikitnya 90% bingkai berada di bawah -50 dBFS
Level rendah	Level keseluruhan rekaman di bawah -35 dBFS
Clipping	Sedikitnya 1% sampel mencapai amplitudo $\geq 0{,}999$
Durasi singkat	Durasi rekaman kurang dari $3{,}75$ detik

Keempat kondisi ini bersifat tidak saling meniadakan (satu rekaman dapat memenuhi lebih dari satu kondisi) dan tidak menggagalkan proses deteksi; peringatan yang dihasilkan hanya ditambahkan pada elemen keluaran "Peringatan mutu" (Tabel

IV.2) agar pengguna memahami konteks hasil, sejalan dengan prinsip pada Subsubbab IV.1.2 bahwa rekaman bermutu rendah tetap diproses, bukan ditolak.

IV.3.12 Implementasi Layanan, Antarmuka, dan Pelaporan

Modul `service.py` mengikat seluruh modul pada Subsubbab IV.3.9 sampai IV.3.11 menjadi satu fungsi orkestrasi `analyze_audio()`, yang dipanggil oleh antarmuka pada `ui.py` setiap kali pengguna mengunggah rekaman. Fungsi ini menjalankan Tahap 1 sampai 7 pada Gambar IV.1 secara berurutan dan mengembalikan satu struktur hasil yang memuat label, skor, peringatan mutu, dan metadata yang diperlukan untuk pelaporan.

Antarmuka pengguna dibangun dengan Streamlit (`ui.py`, `streamlit_app.py` sebagai titik masuk) sesuai rancangan keluaran pada Tabel IV.2. Dua keputusan implementasi berikut relevan dengan prinsip desain pada Subsubbab IV.1.1:

- **Pemuatan model satu kali.** Fungsi pemuat model didekorasi dengan `@st.cache_resource` sehingga `ModelRuntime` (Subsubbab IV.3.10) hanya dimuat dan diverifikasi integritasnya sekali per siklus hidup proses server, bukan setiap kali pengguna mengunggah rekaman – memenuhi pertimbangan efisiensi sumber daya pada Subbab II.3.1.
- **Penanganan kesalahan yang menjaga privasi.** Pesan kesalahan yang ditampilkan kepada pengguna (misalnya saat validasi audio gagal) ditulis secara umum dan tidak menyertakan jejak kesalahan (*stack trace*) atau nama berkas asli, sejalan dengan prinsip pemisahan tanggung jawab klien dan server pada Subsubbab IV.1.1 serta KNF-05.

Setelah hasil ditampilkan, modul `reporting.py` menyediakan dua fungsi pembangkit ringkasan terunduh sesuai Tabel IV.2: `to_json()` menghasilkan berkas JSON terstruktur (nama berkas, durasi, waktu pemeriksaan, *hash* SHA-256 rekaman, label, skor, versi model, dan keterangan batasan), sedangkan `to_text()` menghasilkan ringkasan yang sama dalam bentuk teks naratif yang dapat dibaca langsung oleh pendamping pelaporan (KF-11, KP-09). Sejalan dengan KF-12 dan KNF-05, baik berkas unggahan maupun data antara (waveform dan mel-spectrogram sementara) tidak pernah dituliskan ke penyimpanan permanen di luar sesi pemrosesan yang sedang berjalan.

IV.3.13 Implementasi Evaluasi Purwarupa dan Reproduksi *Baseline*

Untuk memastikan bahwa artefak model yang disertakan pada aplikasi benar-benar identik perilakunya dengan model yang dilatih pada notebook (KNF-08), modul `evaluation.py` menyediakan dua fungsi. Fungsi `binary_metrics()` menghitung

akurasi, presisi/*recall*/F1, matriks konfusi, ROC-AUC, dan EER atas sepasang label dan skor, sedangkan `evaluate_artifacts()` memuat ulang `test.npz` serta model dan konfigurasi tersimpan, menjalankan inferensi, lalu membandingkan matriks konfusi dan ROC-AUC yang dihasilkan terhadap nilai acuan yang diambil langsung dari notebook (Tabel IV.11 dan IV.12); perbedaan di luar toleransi numerik yang kecil akan menggagalkan pemeriksaan ini, sehingga pemeriksaan berfungsi sebagai uji reproduksi *baseline*, bukan sekadar evaluasi ulang. Skrip `scripts/evaluate_model.py` menyediakan antarmuka baris perintah bagi kedua fungsi tersebut.

EER dihitung melalui interpolasi linear pada kurva ROC (*false positive rate* terhadap *false negative rate*) untuk mencari titik potong kedua kurva tersebut, sesuai rumusan pada Persamaan II.7. Nilai inilah yang telah disertakan pada baris EER Tabel IV.11 dan IV.12 pada Subsubbab IV.3.7: EER sebesar $0{,}99\%$ pada tingkat potongan dan $1{,}43\%$ pada tingkat berkas. Kedua nilai ini diperoleh dengan menjalankan ulang model dan data uji yang identik dengan notebook, dan menghasilkan matriks konfusi serta ROC-AUC yang sama persis dengan Tabel IV.11 dan IV.12, sehingga EER dapat dianggap sebagai pelengkap metrik yang sudah ada tanpa mengubah kesimpulan evaluasi sebelumnya.

IV.3.14 Pengujian Perangkat Lunak

Mengikuti pemisahan tanggung jawab antarmodul pada Tabel IV.3, aplikasi disertai rangkaian pengujian otomatis menggunakan `pytest` yang dikelompokkan melalui *marker* menjadi empat kategori, dirangkum pada Tabel IV.14.

Tabel IV.14 Kategori Pengujian Otomatis Aplikasi ADADeH

Kategori	**Lingkup Pengujian**
Unit	Menguji setiap fungsi modul (Tabel IV.3) secara terisolasi, misalnya `assess_quality()` dan `classify_scores()`
Kesetaraan (*parity*)	Membandingkan keluaran fungsi pada paket `adadeh` terhadap fungsi setara pada notebook (misalnya `extract_features` terhadap `gen_melspec_data`, Kode Program IV.4) agar tidak ada penyimpangan perilaku akibat penulisan ulang kode
Integrasi	Menguji `analyze_audio()` (`service.py`) secara menyeluruh, dari berkas audio masukan hingga hasil akhir
Antarmuka (UI)	Menguji elemen antarmuka Streamlit pada `ui.py`

Keberadaan kategori kesetaraan secara khusus menjadi bukti tambahan – selain

reproduksi *baseline* pada Subsubbab IV.3.13 – bahwa penyesuaian implementasi dari notebook ke paket `adadeh` (seperti pemecahan `gen_melspec_data` menjadi `preprocessing.py` yang dapat diuji terpisah) tidak secara tidak sengaja mengubah hasil pemrosesan audio maupun keputusan model. Sejalan dengan README pengembangan aplikasi, status pengujian ini merupakan *checkpoint* pengembangan yang sudah mencakup keempat kategori di atas, namun belum disertai satu kali jalur pengujian penuh (*full test suite*) dan pemeriksaan keamanan yang selesai tanpa kendala waktu jalan (*timeout*), sehingga belum diklaim sebagai rilis produksi (lihat pembatasan pada Subbab IV.4).

IV.4 Rangkuman

Bab ini telah menjabarkan desain sistem ADADeH yang menerjemahkan kebutuhan fungsional dan nonfungsional pada Bab III menjadi alur proses dan rancangan keluaran yang konkret (Subbab IV.1), arsitektur klien–server beserta susunan paket perangkat lunak `adadeh` dan model CNN yang menjalankannya (Subbab IV.2), dan implementasi purwarupa yang telah dikerjakan secara lengkap, mulai dari akuisisi data dan pelatihan model pada notebook hingga aplikasi Streamlit yang telah dibangun dan dapat dijalankan pengguna (Subbab IV.3). Model CNN yang dihasilkan mencapai akurasi dan F1 di atas 99% pada data uji, melampaui target minimal 90% pada Bab I, dengan model yang ringan (sekitar 110 ribu parameter) sesuai pertimbangan sumber daya pada Subbab III.3; EER yang dihitung kemudian melalui modul evaluasi aplikasi (Subsubbab IV.3.13) sebesar $0{,}99\%$ pada tingkat potongan dan $1{,}43\%$ pada tingkat berkas memperkuat kesimpulan bahwa kedua kelas dapat dipisahkan dengan sangat baik.

Antarmuka Streamlit beserta integrasinya dengan modul inferensi – yang pada versi sebelumnya bab ini masih berstatus belum diimplementasikan – telah dibangun secara nyata sebagai paket Python `adadeh` (Subbab IV.2.2) lengkap dengan validasi dan pengerasan keamanan audio, verifikasi integritas artefak model, pemeriksaan mutu rekaman, serta pelaporan hasil yang dapat diunduh (Subsubbab IV.3.9 sampai IV.3.14). Meskipun demikian, sejalan dengan dokumentasi pengembangan aplikasi, purwarupa ini tetap berstatus *checkpoint* pengembangan dan bukan rilis produksi, dengan sejumlah pembatasan yang menjadi acuan pekerjaan pada Bab V:

- Masukan hanya berupa unggahan satu berkas melalui peramban; perekaman langsung melalui mikrofon belum tersedia.
- Analisis hanya mencakup maksimum $K{=}4$ potongan pertama (sekitar 6,25 detik pertama rekaman); rekaman yang lebih panjang belum dianalisis secara menyeluruh, dan aplikasi menampilkan peringatan eksplisit mengenai keter-

batasan cakupan ini kepada pengguna.

- Risiko penggunaan memori berlebih (*out-of-memory*) pada proses *resampling* untuk *sampling rate* masukan yang tidak umum belum terselesaikan, sehingga aplikasi belum direkomendasikan untuk diakses publik atau menerima unggahan dari sumber yang tidak dipercaya tanpa pengawasan tambahan.
- Pengujian otomatis pada peramban (Tabel IV.14, kategori UI) belum selesai secara menyeluruh, dan satu kali jalur pengujian penuh beserta pemeriksaan keamanan belum berhasil diselesaikan tanpa *timeout*, sehingga belum dapat diklaim sebagai hasil pengujian yang bersih secara penuh.

Keempat pembatasan di atas bersifat jujur dan dapat ditindaklanjuti, bukan kegagalan desain: keempatnya berasal dari keputusan sadar untuk membatasi cakupan purwarupa demi keamanan dan kesederhanaan implementasi pada tahap ini, sejalan dengan sifat iteratif pendekatan CRISP-DM pada Bab I.

BAB V

RENCANA SELANJUTNYA

Jelaskan secara detail langkah-langkah rencana selanjutnya, hal-hal yang diperlukan atau akan disiapkan, dan risiko dan mitigasinya, yang meliputi:

1. Rencana implementasi, termasuk alat dan bahan yang diperlukan, lingkungan, konfigurasi, biaya, dan sebagainya.
2. Desain pengujian dan evaluasi, misalnya metode verifikasi dan validasi.
3. Analisis risiko dan mitigasi, misalnya tindakan selanjutnya jika ada yang tidak berjalan sesuai rencana.

DAFTAR PUSTAKA

Bandung, Y. (2025). *Inovasi komputasi multimedia internet untuk pengiriman data berkualitas dan efisien* [Forum Guru Besar Institut Teknologi Bandung. Disampaikan di Aula Barat ITB, 13 September 2025]. ITB Press.

Brandenburg, K. (1999). MP3 and AAC Explained. *Proceedings of the AES 17th International Conference on High-Quality Audio Coding*.

Furht, B. (1994). Multimedia systems: An overview. *IEEE MultiMedia*, *1*(1), 47–59. https://doi.org/10.1109/93.295268

Gibbs, J. W. (1899). Fourier's series. *Nature*, *59*(1539), 606–606.

Iman, M. (2026, May 26). Penyalahgunaan deepfake & voice cloning terhadap keamanan digital dalam perspektif hukum. Retrieved September 28, 2026, from https : / / dandapala . com / opini / detail / penyalahgunaan - deepfake - voice - cloning-terhadap-keamanan-digital-dalam-perspektif-hukum

Li, Z.-N., Drew, M. S., & Liu, J. (2021). *Fundamentals of multimedia* (3rd edisi). Springer. https://doi.org/10.1007/978-3-030-62124-7

McLoughlin, I., Pham, L., Song, Y., Miao, X., Phan, H., Cai, P., Gu, Q., Nan, J., Song, H., & Soh, D. (2026). Spectrogram features for audio and speech analysis. *Applied Sciences*, *16*(2). https://doi.org/10.3390/app16020572

McLoughlin, I. V. (2016). *Speech and audio processing: A matlab-based approach*. Cambridge University Press.

Shannon, C. E. (1949). Communication in the presence of noise. *Proceedings of the IRE*, *37*(1), 10–21. https://doi.org/10.1109/JRPROC.1949.232969

SIP Law Firm. (2025, December 5). Fenomena penipuan digital berupa voice cloning berbasis ai. Retrieved September 28, 2026, from https://siplawfirm.id/resources/fenomena-penipuan-digital-berupa-voice-cloning-berbasis-ai

Stevens, S. S., Volkmann, J., & Newman, E. B. (1937). A scale for the measurement of the psychological magnitude pitch. *The Journal of the Acoustical Society of America*, *8*(3), 185–190. https://doi.org/10.1121/1.1915893

van Beurden, M., & Weaver, A. (2024). *Free Lossless Audio Codec (FLAC)* (RFC No. 9639). Internet Engineering Task Force.

Wikipedia Contributors. (2026, September). *Hearing range*. Wikimedia Foundation. Retrieved October 7, 2026, from https://en.wikipedia.org/wiki/Hearing_range

Winer, E., & Lawrence, M. (2025). *The audio expert: Everything you need to know about audio*. Focal Press.

LAMPIRAN A

SOURCE CODE

A.1 Perangkat Lunak untuk Akuisisi Data dari Sensor Ultrasonik

Listing A.1 Binary search code

```
% -- Example of pseudocode and source code listing --
% -- Gunakan minipage agar listing tidak terpotong ke halaman berikutnya --



\begin{minipage}{\textwidth}
\begin{lstlisting}[caption={Iterative binary search in Python}, label={lst:binary_iter}, frame=single, captionpos=t, language=Python]
def binary_search(arr, target):
    """
    Performs binary search on a sorted array.

    Args:
        arr: A sorted list of elements
        target: The element to search for

    Returns:
        Index of target if found, -1 otherwise
    """
    left = 0
    right = len(arr) - 1

    while left <= right:
        mid = (left + right) // 2

        if arr[mid] == target:
            return mid
        elif arr[mid] < target:
            left = mid + 1
        else:
```

```
            right = mid - 1

    return -1


\end{lstlisting}
\end{minipage}
```

A.2 Perangkat Lunak untuk Pengolahan Data Akuisisi dan Visualisasi Hasil Pengukuran Jarak

asdjfkjashdkjfas